

INSTITUTO INFNET

LUCAS GABRIEL DE OLIVEIRA PORDEUS CAMPOS

ENTREGA FINAL - PROJETO SEMAFORO INTELIGENTE
PROJETO DE BLOCO: SISTEMAS DIGITAIS EMBARCADOS

Manaus/AM

2026

LUCAS GABRIEL DE OLIVEIRA PORDEUS CAMPOS

ENTREGA FINAL - PROJETO SEMAFORO INTELIGENTE

PROJETO DE BLOCO: SISTEMAS DIGITAIS EMBARCADOS

Trabalho apresentado como requisito parcial para a disciplina Projeto de Bloco do curso de Engenharia de Computação.

Professor: Dácio Moreira de Souza

Manaus/AM

2026

SUMÁRIO

Sumário: no Word, clique com o botão direito aqui e escolha “Atualizar campo”.

1. INTRODUÇÃO

Este relatório consolida a Entrega Final do projeto **Semáforo Inteligente com Botão de Pedestre**, um sistema embarcado híbrido formado por uma FPGA **Tang Nano 4K** (Gowin GW1NSR-LV4C, descrita em Verilog) e uma **Raspberry Pi Zero 2W** (programada em Assembly ARM 64 bits, sem biblioteca C). O cruzamento funciona com travessia sob demanda: por padrão o sinal fica verde para os carros; o pedestre aperta um botão para pedir a travessia; e a FPGA mede o fluxo de veículos com um sensor infravermelho — com fluxo baixo o pedestre é liberado mais rápido, com fluxo alto ele espera mais.

Na entrega final o projeto foi revisado com **foco no hardware físico**. As principais decisões desta etapa foram:

- **Raspberry Pi apenas como monitor:** o protocolo serial passou a ser somente de telemetria (FPGA → ARM). Nenhum parâmetro do semáforo pode ser alterado pela Raspberry; os tempos são parâmetros fixos do Verilog.
- **Sem dados simulados:** foram removidos o modo de GPIO simulado, o monitor com telemetria fictícia e os programas de benchmark que mediam transações sem hardware. Tudo o que o sistema mostra vem do sensor, do botão ou da lógica da FSM.
- **Pinagem revista por banco de tensão:** sensor e LEDs saíram dos pinos do HDMI (banco de 2,5 V) para bancos de 3,3 V.
- **Debounce em milissegundos, sincronizadores de 2 flip-flops e reset sincronizado** adequados a sinais mecânicos e a entradas vindas de outra placa.
- **BRAM e DSP usados de fato:** a BRAM guarda o histórico de fluxo lido pela média móvel e a divisão por 5 usa o multiplicador dedicado — ambos confirmados no relatório de place & route.
- **Fluxo afetando a espera do pedestre em qualquer momento:** o tempo de verde passou a valer também a partir do apertado do botão, e não só na entrada da fase.

1.1 Organização do repositório

Tabela 1 – Estrutura do repositório entregue

Pasta	Conteúdo
final_project/rtl/	11 módulos Verilog; topo top_semaforo.v
final_project/tb/	10 testbenches auto-verificáveis, um por módulo
final_project/constraints/	tangnano4k.cst (pinagem) e tangnano4k.sdc (clock de 27 MHz)
final_project/asm/	monitor_semaforo.s e biblioteca lib/ (GPIO, driver serial, telemetria, strings)
final_project/docs/evidencias/	logs de simulação, síntese e montagem
final_project/Makefile	alvos sim, sim-<módulo>, synth, asm, run, clean
TP1/ ... TP5/	entregas incrementais mantidas como registro (inclui NEON, benchmark e parser do TP3/TP4)
RELATORIOS/	relatórios de cada etapa e este relatório final

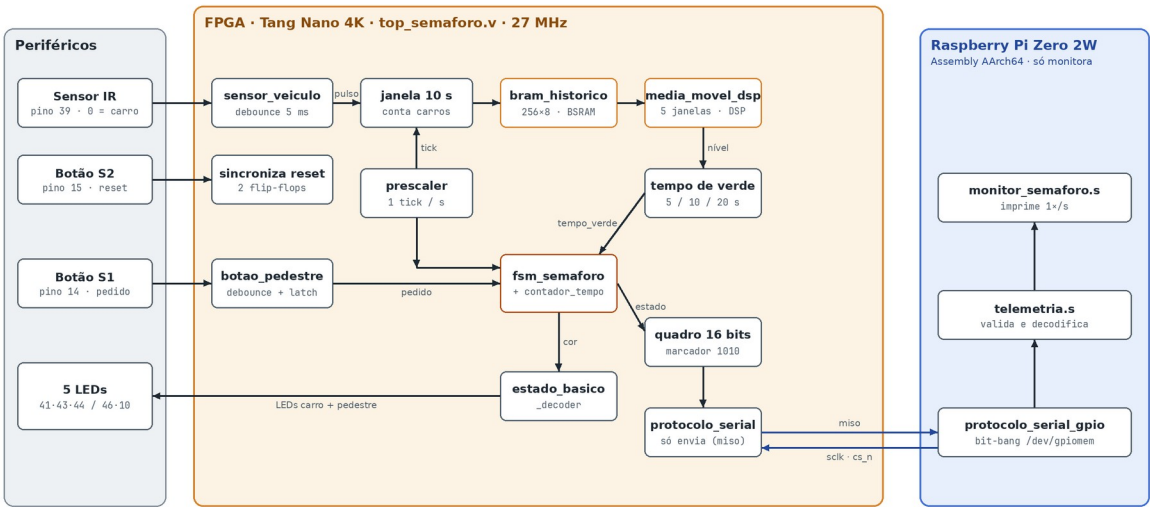
Fonte: elaborado pelo autor (2026).

2. ARQUITETURA DO SISTEMA (ARM-FPGA)

2.1 Visão geral

A Figura 1 mostra o sistema completo. Toda a decisão de tempo real fica na FPGA: o sensor gera um pulso por carro, os pulsos são contados em janelas de 10 s, cada contagem é gravada na BRAM, a média das últimas 5 janelas é calculada no bloco DSP e classificada em fluxo baixo, médio ou alto, que define o tempo de verde usado pela FSM. A FSM acende os LEDs e alimenta o quadro de telemetria que a Raspberry Pi lê pelo link serial.

Figura 1 – Diagrama de blocos do sistema completo



Fonte: elaborado pelo autor (2026).

2.2 Entradas, saídas e pinagem

A Tang Nano 4K tem bancos de I/O com tensões diferentes (banco 0 e 1 em 3,3 V, banco 2 em 2,5 V e banco 3 em 1,8 V, conforme o pinout oficial da Sipeed). Por isso cada sinal foi colocado num banco compatível com o nível elétrico do dispositivo ligado a ele (Tabela 2).

Tabela 2 – Entradas e saídas da FPGA (constraints/tangnano4k.cst)

Sinal	Pino	Banco	Direção	Função
clk	45	1 (3,3 V)	entrada	oscilador de 27 MHz da placa
rst_n	15	3 (1,8 V)	entrada	botão S2 da placa: reset (0 = apertado)
botao_raw	14	3 (1,8 V)	entrada	botão S1 da placa: pedido de travessia
sensor_raw	39	1 (3,3 V)	entrada	sensor IR (0 = veículo), pull-up
led_vermelho/ amarelo/verde	41 / 43 / 44	1 (3,3 V)	saída	LEDs dos carros
led_ped_verde/ vermelho	46 / 10	1 / 0 (3,3 V)	saída	LEDs do pedestre
sclk / cs_n	40 / 42	1 (3,3 V)	entrada	clock e seleção de quadro vindos da Raspberry
miso	33	2 (2,5 V)	saída	telemetria para o GPIO20 da Raspberry

Fonte: elaborado pelo autor (2026).

Do lado da Raspberry Pi são usados apenas três GPIOs e o GND: GPIO11 (pino físico 23) → `sc1k`, GPIO6 (pino 31) → `cs_n` e GPIO20 (pino 38) ← `miso`. O sensor, os botões e os LEDs ficam ligados diretamente na FPGA.

2.3 Divisão de responsabilidades

Tabela 3 – Divisão de responsabilidades entre FPGA e ARM

Aspecto	FPGA (Tang Nano 4K)	ARM (Raspberry Pi)
Faz	lê sensor e botão, filtra ruído, mede fluxo, decide as fases, acende os LEDs, envia telemetria	lê a telemetria, valida o quadro e imprime o estado
Natureza	determinística, decisão a cada ciclo de 37 ns	best-effort, sob Linux
Crítica para segurança	sim	não
Se desligar	—	o semáforo continua idêntico: o link é só de saída

Fonte: elaborado pelo autor (2026).

A justificativa é de tempo real: o Linux tem latência imprevisível (escalonador, interrupções, outros processos) e a decisão “posso fechar o verde?” precisa ser garantida em todo ciclo. Na FPGA a lógica é cabeada e não compete com nada; ao ARM ficou o que se beneficia de flexibilidade de software — visualização e registro. Essa separação implementa a regra de segurança **ONF-08**: o trânsito nunca é interrompido sem pedido e o funcionamento nunca depende do ARM.

2.4 Requisitos de tempo real: clock, BRAM, DSP, PLL e banda

Tabela 4 – Recursos de tempo real e como foram tratados

Recurso	Uso no projeto	Evidência
Clock	oscilador único de 27 MHz; <code>prescaler.v</code> gera 1 tick por segundo	timing: 198,69 MHz máximo (PASS a 27 MHz)
BRAM	histórico circular 256 × 8 com as contagens de cada janela de 10 s	place & route: BSRAM 1/10
DSP	multiplicação soma × 13107 (1/5 em ponto fixo Q16) na média móvel	place & route: MULT18X18 1/16
PLL	não usado: nenhuma parte exige outro clock e 1 s é obtido por divisor digital	PLLVR 0/2
Banda	1 quadro de 16 bits por segundo; <code>sc1k</code> na ordem de 10 kHz gerado pela Raspberry	≈ 16 bit/s de carga útil

Fonte: elaborado pelo autor (2026).

3. HARDWARE EM VERILOG

O hardware tem 11 módulos (Tabela 5). Todos usam reset assíncrono ativo em nível baixo, liberado de forma síncrona no topo, e cada arquivo começa com um comentário que explica o papel do módulo e suas portas.

Tabela 5 – Módulos Verilog do sistema final

Módulo	Tipo	Função
top_semaforo.v	topo	sincroniza o reset, conta veículos por janela, escolhe o tempo de verde, monta o quadro
debounce.v	sequencial	2 flip-flops de sincronização + filtro de N ciclos estáveis
sensor_veiculo.v	sequencial	debounce de 5 ms + 1 pulso por veículo
botao_pedestre.v	sequencial	debounce de 20 ms + latch do pedido, apagado pela FSM
prescaler.v	sequencial	27 MHz → 1 tick por segundo
contador_tempo.v	sequencial	temporizador decrescente (carrega, decrementa, zerou)
fsm_semaforo.v	FSM Moore	3 estados de fase; usa contador_tempo
estado_basico_decoder.v	combinacional	cor de 2 bits → 3 LEDs; código inválido acende vermelho
bram_historico.v	memória	histórico circular 256 × 8 inferido em BSRAM
media_movel_dsp.v	aritmético	média de 5 amostras com multiplicação Q16 no DSP
protocolo_serial.v	interface	escravo SPI modo 0, só envia (16 bits por quadro)

Fonte: elaborado pelo autor (2026).

3.1 Condicionamento das entradas

Botão e sensor são sinais mecânicos/analógicos e assíncronos ao clock. O `debounce.v` (Figura 2) resolve os dois problemas: a entrada passa por **dois flip-flops** (`sinc`) antes de ser usada, o que dá um ciclo inteiro para uma eventual metaestabilidade se resolver; depois, a saída só muda quando a entrada sincronizada fica `N_CICLOS` ciclos seguidos no novo nível — qualquer repique zera o contador. No hardware real são 540 000 ciclos (20 ms) para o botão e 135 000 (5 ms) para o sensor.

Figura 2 – Filtro de debounce com sincronizador de 2 flip-flops

```
rtl/debounce.v · linhas 12-42

12 module debounce #(
13     parameter integer N_CICLOS = 540_000
14 ) (
15     input wire clk,
16     input wire rst_n,
17     input wire entrada,
18     output reg saida
19 );
20 localparam integer LARGURA_CONT = $clog2(N_CICLOS + 1);
21
22 reg [1:0] sinc; // cadeia de sincronizacao; sinc[1] = entrada ja sincronizada
23 reg [LARGURA_CONT-1:0] contador; // ciclos seguidos em que sinc[1] difere de saida
24
25 always @(posedge clk or negedge rst_n) begin
26     if (!rst_n) begin
27         sinc <= 2'b00;
28         contador <= {LARGURA_CONT{1'b0}};
29         saida <= 1'b0;
30     end else begin
31         sinc <= {sinc[0], entrada};
32
33         if (sinc[1] == saida)
34             contador <= {LARGURA_CONT{1'b0}}; // qualquer repique reinicia a contagem
35         else if (contador == N_CICLOS - 1) begin
36             saida <= sinc[1]; // estavel tempo suficiente: aceita o novo nivel
37             contador <= {LARGURA_CONT{1'b0}};
38         end else
39             contador <= contador + 1'b1;
40     end
41 end
42 endmodule
```

Fonte: elaborado pelo autor (2026).

O sensor_veiculo.v transforma o nível filtrado em **um pulso de 1 ciclo por veículo**, detectando a borda de subida (Figura 3). Assim um carro que fica centenas de milissegundos na frente do sensor conta uma única vez. O botao_pedestre.v usa a mesma borda para **travar o pedido** num latch (Figura 4); o pedido continua ativo depois que o botão é solto e só é apagado por limpa_solicitacao, gerado pela FSM no fim da travessia.

Figura 3 – Sensor de veículo: debounce e detecção de borda

```
rtl/sensor_veiculo.v · linhas 12-36

12 module sensor_veiculo #(
13     parameter integer N_CICLOS = 135_000
14 ) (
15     input wire clk,
16     input wire rst_n,
17     input wire sensor_ativo,
18     output wire veiculo_pulso
19 );
20 wire sensor_estavel; // deteccao filtrada (1 enquanto ha veiculo na frente do sensor)
21 reg estavel_ant; // sensor_estavel no ciclo anterior, para detectar a borda
22
23 debounce #(.N_CICLOS(N_CICLOS)) u_debounce (
24     .clk(clk), .rst_n(rst_n),
25     .entrada(sensor_ativo), .saida(sensor_estavel)
26 );
27
28 always @(posedge clk or negedge rst_n) begin
29     if (!rst_n)
30         estavel_ant <= 1'b0;
31     else
32         estavel_ant <= sensor_estavel;
33 end
34
35 assign veiculo_pulso = sensor_estavel & ~estavel_ant;
36 endmodule
```

Fonte: elaborado pelo autor (2026).

Figura 4 – Latch do pedido de travessia

```
rtl/botao_pedestre.v · linhas 30-49

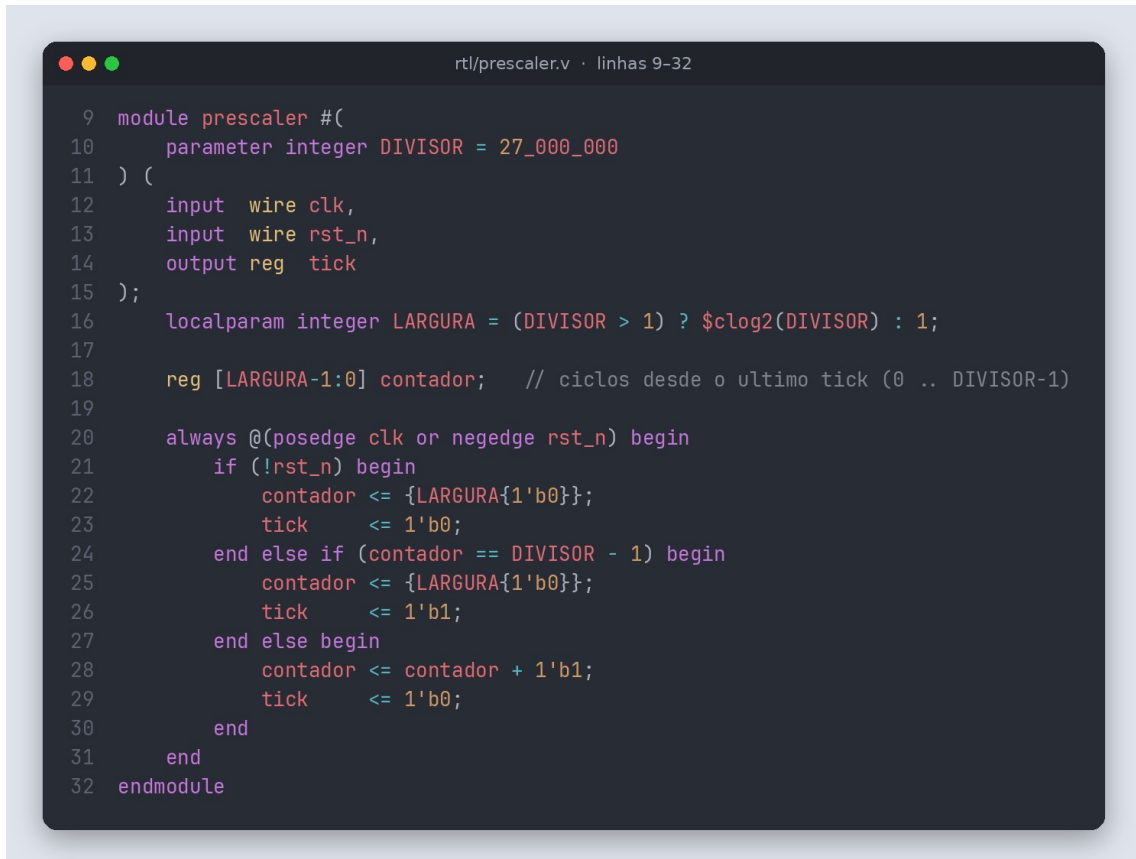
30     always @(posedge clk or negedge rst_n) begin
31         if (!rst_n)
32             estavel_ant <= 1'b0;
33         else
34             estavel_ant <= botao_estavel;
35     end
36
37     assign pressionado_pulso = botao_estavel & ~estavel_ant;
38
39     // limpa tem prioridade: um aperto no mesmo ciclo do fim da fase de
40     // pedestre e' descartado (a travessia acabou de acontecer)
41     always @(posedge clk or negedge rst_n) begin
42         if (!rst_n)
43             solicitacao_pedestre <= 1'b0;
44         else if (limpa_solicitacao)
45             solicitacao_pedestre <= 1'b0;
46         else if (pressionado_pulso)
47             solicitacao_pedestre <= 1'b1;
48     end
49 endmodule
```

Fonte: elaborado pelo autor (2026).

3.2 Base de tempo

O `prescaler.v` conta ciclos de 27 MHz e gera um pulso `tick` a cada 27 000 000 ciclos (1 s). O `contador_tempo.v` é um contador decrescente com carga prioritária: carrega a duração da fase, desconta 1 a cada tick e para em zero, sinalizando zero (Figuras 5 e 6). Nos testbenches o divisor é reduzido para que um “segundo” dure poucos ciclos.

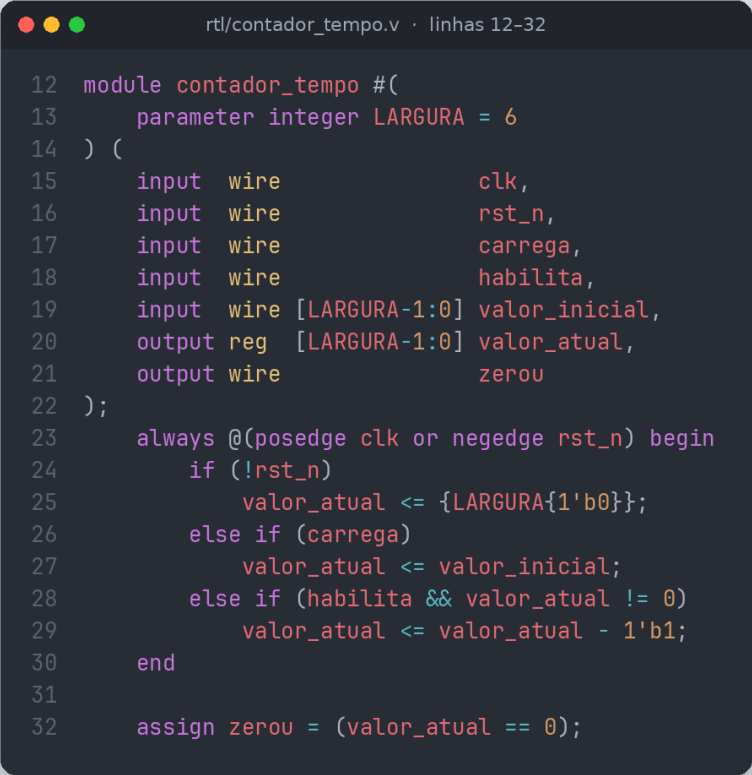
Figura 5 – Prescaler: 27 MHz para 1 tick por segundo

A screenshot of a code editor window titled "rtl/prescaler.v · linhas 9-32". The code is Verilog for a prescaler module. It defines a module with two inputs: "clk" (wire) and "rst_n" (wire), and one output: "tick" (reg). A parameter "DIVISOR" is set to 27_000_000. A local parameter "LARGURA" is calculated as the ceiling of log2(DIVISOR), which is 1. A register "contador" of width LARGURA is declared, with a comment indicating it counts cycles from the last tick (0 to DIVISOR-1). The logic is implemented in an "always" block triggered by the clock edge or reset. On reset, the counter is loaded with 0 and tick is 0. When the counter reaches DIVISOR-1, it is reset to 0 and tick is set to 1. Otherwise, the counter is incremented by 1 and tick remains 0.

```
9  module prescaler #(
10     parameter integer DIVISOR = 27_000_000
11 ) (
12     input  wire clk,
13     input  wire rst_n,
14     output reg  tick
15 );
16     localparam integer LARGURA = (DIVISOR > 1) ? $clog2(DIVISOR) : 1;
17
18     reg [LARGURA-1:0] contador; // ciclos desde o ultimo tick (0 .. DIVISOR-1)
19
20     always @(posedge clk or negedge rst_n) begin
21         if (!rst_n) begin
22             contador <= {LARGURA{1'b0}};
23             tick      <= 1'b0;
24         end else if (contador == DIVISOR - 1) begin
25             contador <= {LARGURA{1'b0}};
26             tick      <= 1'b1;
27         end else begin
28             contador <= contador + 1'b1;
29             tick      <= 1'b0;
30         end
31     end
32 endmodule
```

Fonte: elaborado pelo autor (2026).

Figura 6 – Contador decrescente usado pela FSM



```
rtl/contador_tempo.v · linhas 12-32

12 module contador_tempo #(
13     parameter integer LARGURA = 6
14 ) (
15     input wire        clk,
16     input wire        rst_n,
17     input wire        carrega,
18     input wire        habilita,
19     input wire [LARGURA-1:0] valor_inicial,
20     output reg [LARGURA-1:0] valor_atual,
21     output wire       zerou
22 );
23 always @(posedge clk or negedge rst_n) begin
24     if (!rst_n)
25         valor_atual <= {LARGURA{1'b0}};
26     else if (carrega)
27         valor_atual <= valor_inicial;
28     else if (habilita && valor_atual != 0)
29         valor_atual <= valor_atual - 1'b1;
30 end
31
32 assign zerou = (valor_atual == 0);
```

Fonte: elaborado pelo autor (2026).

3.3 Contagem de fluxo e histórico em BRAM

No topo, os pulsos de veículo são acumulados em `contagem_janela`; a cada `JANELA_AMOSTRAGEM` ticks (10 s) a janela fecha, o total vira `amostra_fluxo` e `nova_amostra` pulsa por um ciclo (Figura 7). Um carro que passe exatamente no ciclo de fechamento entra na amostra entregue (`contagem_com_pulso`), e o contador satura em 255.

Figura 7 – Contagem de veículos por janela de 10 s (top_semaforo.v)

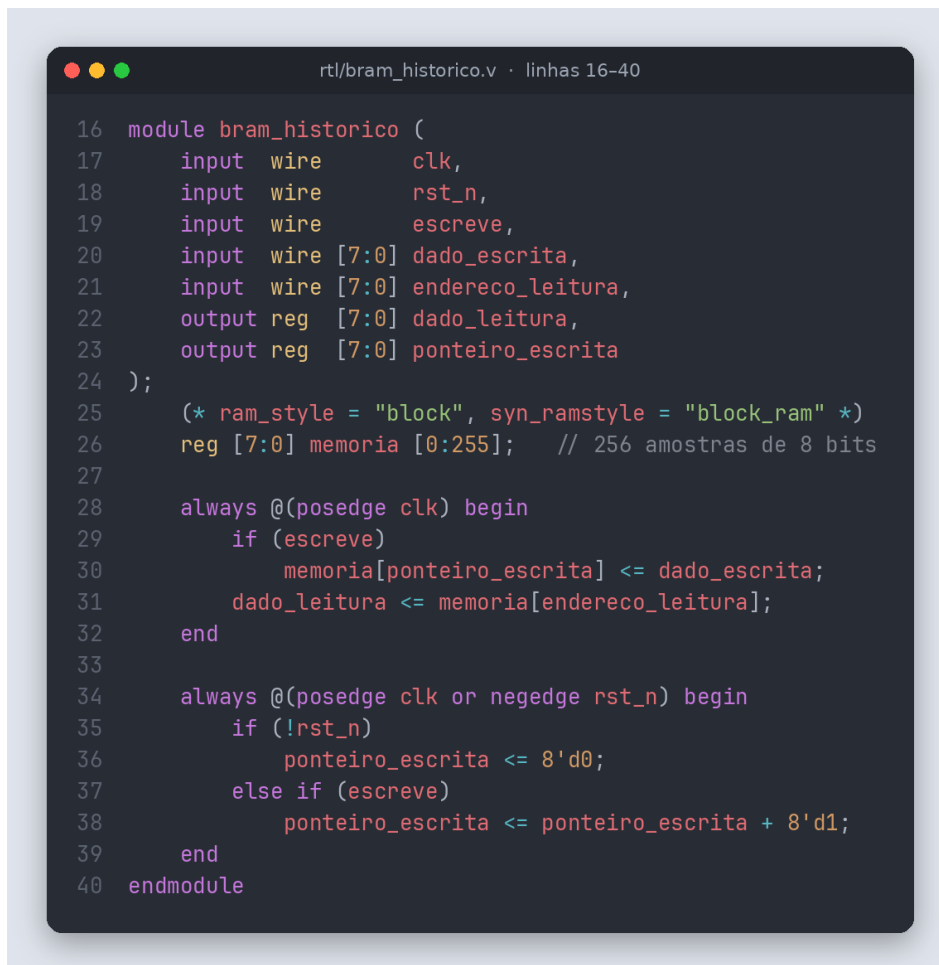
```
rtl/top_semaforo.v · linhas 96-127

96 // ---- contagem de veículos por janela de JANELA_AMOSTRAGEM segundos ----
97 localparam integer LARGURA_JANELA = (JANELA_AMOSTRAGEM > 1) ? $clog2(JANELA_AMOSTRAGEM) : 1;
98
99 reg [LARGURA_JANELA-1:0] ticks_janela; // segundos ja passados na janela atual
100 reg [7:0] contagem_janela; // veiculos na janela atual (satura em 255)
101 reg [7:0] amostra_fluxo; // contagem da ultima janela fechada
102 reg nova_amostra; // 1 ciclo em 1 quando amostra_fluxo e' atualizada
103
104 wire [7:0] contagem_com_pulso = (veiculo_pulso && contagem_janela != 8'hFF)
105                               ? contagem_janela + 8'd1 : contagem_janela;
106
107 always @(posedge clk or negedge rst_sist_n) begin
108     if (!rst_sist_n) begin
109         ticks_janela <= {LARGURA_JANELA{1'b0}};
110         contagem_janela <= 8'd0;
111         amostra_fluxo <= 8'd0;
112         nova_amostra <= 1'b0;
113     end else begin
114         nova_amostra <= 1'b0;
115         if (tick && ticks_janela == JANELA_AMOSTRAGEM - 1) begin
116             // fecha a janela: entrega a contagem e comeca outra do zero
117             amostra_fluxo <= contagem_com_pulso;
118             nova_amostra <= 1'b1;
119             contagem_janela <= 8'd0;
120             ticks_janela <= {LARGURA_JANELA{1'b0}};
121         end else begin
122             contagem_janela <= contagem_com_pulso;
123             if (tick)
124                 ticks_janela <= ticks_janela + 1'b1;
125         end
126     end
127 end
```

Fonte: elaborado pelo autor (2026).

A amostra é gravada no `bram_historico.v` (Figura 8): 256 posições de 8 bits, ponteiro de escrita de 8 bits que volta a 0 sozinho (buffer circular) e **leitura síncrona** com um ciclo de atraso. O topo liga a porta de leitura em ponteiro – 5, que é a amostra que sai da média móvel. Os atributos `ram_style/syn_ramstyle` pedem um bloco de memória ao Yosys e ao Gowin EDA.

Figura 8 – Histórico circular em BRAM



```
16 module bram_historico (
17     input wire      clk,
18     input wire      rst_n,
19     input wire      escreve,
20     input wire [7:0] dado_escrita,
21     input wire [7:0] endereco_leitura,
22     output reg [7:0] dado_leitura,
23     output reg [7:0] ponteiro_escrita
24 );
25 (* ram_style = "block", syn_ramstyle = "block_ram" *)
26 reg [7:0] memoria [0:255]; // 256 amostras de 8 bits
27
28 always @(posedge clk) begin
29     if (escreve)
30         memoria[ponteiro_escrita] <= dado_escrita;
31         dado_leitura <= memoria[endereco_leitura];
32 end
33
34 always @(posedge clk or negedge rst_n) begin
35     if (!rst_n)
36         ponteiro_escrita <= 8'd0;
37     else if (escreve)
38         ponteiro_escrita <= ponteiro_escrita + 8'd1;
39 end
40 endmodule
```

Fonte: elaborado pelo autor (2026).

Por que a memória não tem reset: uma BRAM grava um endereço por ciclo; não existe fio que zere as 256 posições de uma vez. Com reset no código, a síntese implementaria a memória com 2 048 flip-flops e abandonaria o bloco BSRAM. Por isso só o ponteiro é zerado; quem impede que o conteúdo antigo entre na conta é o contador de amostras válidas da média (seção 3.4). Ao ligar a placa o bitstream inicializa a BSRAM com zeros; ao apertar S2 as amostras antigas permanecem, mas são ignoradas.

3.4 Média móvel no bloco DSP

O `media_movel_dsp.v` (Figura 9) guarda só a soma da janela; as amostras ficam na BRAM. A cada janela nova faz $\text{soma_nova} = \text{soma} - \text{valor_saindo} + \text{amostra}$, onde `valor_saindo` é a amostra lida da BRAM — ou 0 enquanto `amostras_validas` não chega a 5, o que torna o filtro correto logo após o reset. A divisão por 5 é feita multiplicando pelo recíproco em ponto fixo Q16 ($13107 \approx 65536/5$), somando 0,5 (32768) para arredondar e tomando os bits [23:16] como parte inteira.

Figura 9 – Média móvel com soma incremental e multiplicação Q16

```

rtl/media_movel_dsp.v · linhas 28-66

28  localparam integer N_JANELA      = 5;          // amostras na media
29  localparam [16:0] RECIPROCO_Q16 = 17'd13107;    // round(65536 / 5): 1/5 em ponto fixo Q16
30
31  reg [2:0] amostras_validas;    // amostras ja na janela (satura em N_JANELA)
32  reg [10:0] soma;               // soma da janela (5 x 255 = 1275 cabe em 11 bits)
33  wire [7:0] valor_saindo;       // amostra_saindo, ou 0 enquanto a janela ainda enche
34  wire [10:0] soma_nova;         // soma ja com a amostra que entra e sem a que sai
35  (* syn_dspstyle = "dsp" *)
36  wire [27:0] produto;           // soma_nova * (1/5) em Q16 → bloco DSP
37  wire [27:0] produto_arred;     // produto + 0,5 (arredonda ao descartar a parte fracionaria)
38  wire [7:0] media_nova;         // parte inteira de produto_arred = round(soma_nova / 5)
39
40  assign valor_saindo = (amostras_validas == N_JANELA) ? amostra_saindo : 8'd0;
41  assign soma_nova    = soma - valor_saindo + amostra;
42  assign produto      = soma_nova * RECIPROCO_Q16;
43  assign produto_arred = produto + 28'd32768;
44  assign media_nova   = produto_arred[23:16];
45
46  always @(posedge clk or negedge rst_n) begin
47      if (!rst_n) begin
48          amostras_validas <= 3'd0;
49          soma <= 11'd0;
50          media <= 8'd0;
51          nivel_fluxo <= 2'b00;
52      end else if (nova_amostra) begin
53          if (amostras_validas != N_JANELA)
54              amostras_validas <= amostras_validas + 3'd1;
55          soma <= soma_nova;
56          media <= media_nova;
57
58          if (media_nova <= LIMIAR_BAIXO)
59              nivel_fluxo <= 2'b00;
60          else if (media_nova <= LIMIAR_ALTO)
61              nivel_fluxo <= 2'b01;
62          else
63              nivel_fluxo <= 2'b10;
64      end
65  end
66  endmodule

```

Fonte: elaborado pelo autor (2026).

Exemplo: com soma = 20, saindo 4 e entrando 6, soma_nova = 22; $22 \times 13107 + 32768 = 321\ 122$; bits [23:16] = 4 = round(4,4). O erro da aproximação 13107 em vez de 13107,2 é no máximo 0,004 para a maior soma possível (1 275), pequeno demais para alterar o arredondamento. A média é comparada com os limiares do topo (LIMIAR_BAIXO = 1, LIMIAR_ALTO = 3), o que equivale às faixas da Tabela 6.

Tabela 6 – Classificação do fluxo e tempo de verde

Fluxo	Média por janela de 10 s	Carros nos últimos 50 s	Tempo de verde
Baixo	0 ou 1	0 a 7	5 s
Médio	2 ou 3	8 a 17	10 s
Alto	4 ou mais	18 ou mais	20 s

Fonte: elaborado pelo autor (2026).

Figura 10 – Escolha do tempo de verde pelo nível de fluxo (top_semaforo.v)

```

rtl/top_semaforo.v · linhas 152-161
152 // ---- tempo de verde conforme o fluxo medido ----
153 reg [5:0] tempo_verde; // segundos de verde dos carros para o nível atual
154
155 always @(*) begin
156     case (nivel_fluxo)
157         2'b00: tempo_verde = TEMPO_VERDE_BAIXO;
158         2'b10: tempo_verde = TEMPO_VERDE_ALTO;
159         default: tempo_verde = TEMPO_VERDE_MEDIO;
160     endcase
161 end

```

Fonte: elaborado pelo autor (2026).

3.5 Máquina de estados do semáforo

A fsm_semaforo.v é uma FSM de Moore com três estados (Figura 11). Ela é escrita em três blocos: o registrador de estado, a lógica combinacional de próximo estado e saídas (Figura 12) e a escolha do que carregar no temporizador (Figura 13).

Figura 11 – Diagrama de estados da FSM



Fonte: elaborado pelo autor (2026).

Tabela 7 – Estados da FSM

Estado	Carros	Pedestre	Duração	Sai quando
CARRO_VERDE	verde	vermelho	indefinida	há pedido e o tempo de verde acabou
CARRO_AMARELO	amarelo	vermelho	3 s	o tempo acaba
PEDESTRE_VERDE	vermelho	verde	15 s	o tempo acaba (apaga o pedido)

Fonte: elaborado pelo autor (2026).

Figura 12 – FSM: registrador de estado e lógica de próximo estado

```
rtl/fsm_semaforo.v · linhas 73-117

73     assign pedido_novo = solicitacao_pedestre & ~solicitacao_ant;
74
75     always @(posedge clk or negedge rst_n) begin
76         if (!rst_n) begin
77             estado          <= CARRO_VERDE;
78             iniciou         <= 1'b0;
79             solicitacao_ant <= 1'b0;
80         end else begin
81             estado          <= prox_estado;
82             iniciou         <= 1'b1;
83             solicitacao_ant <= solicitacao_pedestre;
84         end
85     end
86
87     // proximo estado e saidas (Moore: as cores dependem so' de "estado")
88     always @(*) begin
89         prox_estado      = estado;
90         cor_carro        = COR_VERMELHO;
91         pedestre_verde   = 1'b0;
92         limpa_solicitacao = 1'b0;
93
94         case (estado)
95             CARRO_VERDE: begin
96                 cor_carro = COR_VERDE;
97                 // pedido_novo bloqueia a troca no ciclo em que o pedido
98                 // chega, para dar tempo de recarregar a espera do pedestre
99                 if (tempo_esgotado && solicitacao_pedestre && !pedido_novo)
100                     prox_estado = CARRO_AMARELO;
101             end
102             CARRO_AMARELO: begin
103                 cor_carro = COR_AMARELO;
104                 if (tempo_esgotado)
105                     prox_estado = PEDESTRE_VERDE;
106             end
107             PEDESTRE_VERDE: begin
108                 cor_carro      = COR_VERMELHO;
109                 pedestre_verde = 1'b1;
110                 if (tempo_esgotado) begin
111                     prox_estado      = CARRO_VERDE;
112                     limpa_solicitacao = 1'b1;
113                 end
114             end
115             default: prox_estado = CARRO_VERDE; // estado invalido: tudo vermelho por 1 ciclo
116         endcase
117     end
```

Fonte: elaborado pelo autor (2026).

Figura 13 – FSM: carga do temporizador e regra do pedido novo

```
rtl/fsm_semaforo.v · linhas 119-132

119 // carga do contador: duracao da fase que esta COMECANDO (prox_estado),
120 // ou a espera do pedestre quando um pedido novo chega durante o verde
121 always @(*) begin
122     case (prox_estado)
123         CARRO_AMARELO: valor_carga = tempo_amarelo;
124         PEDESTRE_VERDE: valor_carga = tempo_pedestre;
125         default:      valor_carga = tempo_verde;
126     endcase
127
128     carrega_cont = !iniciou
129                 || (prox_estado != estado)
130                 || (estado == CARRO_VERDE && pedido_novo && contagem_atual < tempo_verde);
131 end
132 endmodule
```

Fonte: elaborado pelo autor (2026).

Regra do pedido novo. Sem pedido, o verde fica aceso indefinidamente e o contador fica parado em zero. Se o tempo de verde valesse só na entrada da fase, um aperto depois de minutos liberaria o pedestre na hora, qualquer que fosse o fluxo. Por isso, no ciclo em que o pedido chega (pedido_novo) durante o verde, o contador é recarregado com o tempo de verde se faltar menos que isso; a troca para o amarelo fica bloqueada nesse ciclo para a recarga valer antes. A espera passa a ser o maior entre o que faltava e o tempo de verde (Tabela 8).

Tabela 8 – Espera do pedestre após o aperto

Situação	Faltava	Tempo de verde	Espera
verde aceso há minutos, fluxo baixo	0 s	5 s	5 s
verde aceso há minutos, fluxo alto	0 s	20 s	20 s
pedido 1 s após o verde começar	4 s	5 s	5 s (não soma)
já faltava mais que o tempo de verde	18 s	5 s	18 s

Fonte: elaborado pelo autor (2026).

Segurança. pedestre_verde = 1 só existe no estado PEDESTRE_VERDE, onde cor_carro é vermelho; como as saídas dependem só do estado, nenhuma combinação de entradas acende os dois verdes. Os valores padrão do bloco combinacional são “tudo vermelho”, e o código de estado inválido passa um ciclo nessa condição antes de voltar a CARRO_VERDE.

3.6 Protocolo serial de telemetria

O protocolo_serial.v (Figura 14) é um escravo no estilo SPI modo 0, somente de envio. sclk e cs_n vêm da Raspberry e passam por flip-flops de sincronização; a borda de subida de sclk é detectada com o clock de 27 MHz, sem criar outro domínio de clock. Enquanto cs_n = 1 o registrador copia o quadro atual; quando cs_n desce o quadro é congelado — todos os campos são do mesmo instante — e cada subida de sclk desloca um bit para miso.

Figura 14 – Escravo serial de telemetria

```
rtl/protocolo_serial.v · linhas 30-53

30 reg [2:0] sclk_sinc; // [1:0] sincronizam sclk; [2] guarda o valor anterior para achar a borda
31 reg [1:0] cs_n_sinc; // cs_n sincronizado em cs_n_sinc[1]
32 reg [15:0] deslocamento; // quadro sendo enviado; o bit 15 esta em miso
33 wire sclk_subiu; // 1 ciclo em 1 na borda de subida de sclk
34
35 assign sclk_subiu = sclk_sinc[1] & ~sclk_sinc[2];
36 assign miso = deslocamento[15];
37
38 always @(posedge clk or negedge rst_n) begin
39     if (!rst_n) begin
40         sclk_sinc <= 3'b000;
41         cs_n_sinc <= 2'b11;
42         deslocamento <= 16'd0;
43     end else begin
44         sclk_sinc <= {sclk_sinc[1:0], sclk};
45         cs_n_sinc <= {cs_n_sinc[0], cs_n};
46
47         if (cs_n_sinc[1])
48             deslocamento <= quadro; // fora do quadro: fotografa o estado atual
49         else if (sclk_subiu)
50             deslocamento <= {deslocamento[14:0], 1'b0}; // dentro do quadro: proximo bit
51     end
52 end
53 endmodule
```

Fonte: elaborado pelo autor (2026).

O quadro tem 16 bits (Figura 15 e Tabela 9). O marcador fixo 1010 permite ao ARM descartar leituras ruins: um fio solto, GND ausente ou FPGA não gravada produzem só zeros ou só uns.

Figura 15 – Formato do quadro de telemetria

marcador				cor carros		ped.	pedido	fluxo		segundos restantes					
1	0	1	0	1	0	0	1	0	1	0	0	0	1	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Exemplo 0xA944: carros VERDE · pedestre VERMELHO · pedido pendente · fluxo MÉDIO · 4 s
Enviado MSB primeiro, um bit por subida de sclk, com cs_n em nível baixo.

Fonte: elaborado pelo autor (2026).

Tabela 9 – Campos do quadro de telemetria

Bits	Campo	Valores
15–12	marcador	sempre 1010
11–10	cor dos carros	00 vermelho · 01 amarelo · 10 verde
9	cor do pedestre	1 verde · 0 vermelho
8	pedido pendente	1 sim · 0 não
7–6	nível de fluxo	00 baixo · 01 médio · 10 alto
5–0	segundos restantes da fase	0 a 63

Fonte: elaborado pelo autor (2026).

Figura 16 – Montagem do quadro no topo

```
rtl/top_semaforo.v · linhas 190-208

190 // ---- telemetria para a Raspberry Pi ----
191 // Quadro de 16 bits (MSB primeiro):
192 // [15:12] marcador fixo 1010: a Raspberry descarta quadros sem ele
193 // [11:10] cor dos carros: 00 vermelho, 01 amarelo, 10 verde
194 // [9] pedestre: 1 verde, 0 vermelho
195 // [8] pedido de travessia pendente
196 // [7:6] nível de fluxo: 00 baixo, 01 medio, 10 alto
197 // [5:0] segundos restantes da fase atual
198 localparam [3:0] MARCADOR_QUADRO = 4'b1010;
199
200 wire [15:0] quadro_telemetria = {MARCADOR_QUADRO, cor_carro, pedestre_verde,
201                                  solicitacao_pedestre, nivel_fluxo, contagem_fase};
202
203 protocolo_serial u_telemetria (
204     .clk(clk), .rst_n(rst_sist_n),
205     .sclk(sclk), .cs_n(cs_n),
206     .quadro(quadro_telemetria),
207     .miso(miso)
208 );
```

Fonte: elaborado pelo autor (2026).

3.7 Reset e restrições físicas

O botão S2 entra direto no reset assíncrono, mas a liberação passa por dois flip-flops (Figura 17), para que todos os registradores saiam do reset na mesma borda. A Figura 18 mostra o arquivo de constraints, já organizado por banco de tensão.

Figura 17 – Reset com liberação síncrona

```
rtl/top_semaforo.v · linhas 61-71

61 // ---- reset: entra na hora, sai sincronizado ao clk ----
62 reg [1:0] reset_sinc; // reset_sinc[1] = 1 dois ciclos apos soltar S2
63 wire rst_sist_n; // reset usado por todo o sistema (ativo em 0)
64
65 always @(posedge clk or negedge rst_n) begin
66     if (!rst_n)
67         reset_sinc <= 2'b00;
68     else
69         reset_sinc <= {reset_sinc[0], 1'b1};
70 end
71 assign rst_sist_n = reset_sinc[1];
```

Fonte: elaborado pelo autor (2026).

Figura 18 – Constraints de pinagem (tangnano4k.cst)

```
constraints/tangnano4k.cst · linhas 1-44

1 // tangnano4k.cst: pinagem do semaforo inteligente na Tang Nano 4K
2 // (GW1NSR-LV4CQN48P). Numeros = pinos do chip, impressos no header da placa.
3 //
4 // Bancos de I/O da placa (pinout oficial Sipeed):
5 //   banco 0 = 3,3 V | banco 1 = 3,3 V | banco 2 = 2,5 V (HDMI) | banco 3 = 1,8 V
6 // Sensor, LEDs e o link com a Raspberry Pi (3,3 V) ficam nos bancos 0/1.
7 // Os pinos 1-4, 6-9 e 47-48 (flash, JTAG, DONE) nao sao usados.
8
9 // oscilador de 27 MHz da placa
10 IO_LOC "clk" 45;
11 IO_PORT "clk" IO_TYPE=LVCMS33;
12
13 // botoes da placa (banco 3, 1,8 V, pull-up externo; apertado = 0)
14 IO_LOC "rst_n" 15; // S2 = reset
15 IO_PORT "rst_n" IO_TYPE=LVCMS18 PULL_MODE=UP;
16 IO_LOC "botao_raw" 14; // S1 = pedido de travessia
17 IO_PORT "botao_raw" IO_TYPE=LVCMS18 PULL_MODE=UP;
18
19 // sensor IR de veiculos (saida digital 3,3 V; 0 = veiculo detectado)
20 IO_LOC "sensor_raw" 39;
21 IO_PORT "sensor_raw" IO_TYPE=LVCMS33 PULL_MODE=UP;
22
23 // LEDs dos carros (1 = aceso; resistor de 220 ohm ate o GND)
24 IO_LOC "led_vermelho" 41;
25 IO_PORT "led_vermelho" IO_TYPE=LVCMS33 DRIVE=8;
26 IO_LOC "led_amarelo" 43;
27 IO_PORT "led_amarelo" IO_TYPE=LVCMS33 DRIVE=8;
28 IO_LOC "led_verde" 44;
29 IO_PORT "led_verde" IO_TYPE=LVCMS33 DRIVE=8;
30
31 // LEDs do pedestre; o pino 10 tambem acende o LED da propria placa
32 IO_LOC "led_ped_verde" 46;
33 IO_PORT "led_ped_verde" IO_TYPE=LVCMS33 DRIVE=8;
34 IO_LOC "led_ped_vermelho" 10;
35 IO_PORT "led_ped_vermelho" IO_TYPE=LVCMS33 DRIVE=8;
36
37 // link de telemetria com a Raspberry Pi
38 IO_LOC "sclk" 40; // entrada, vem do GPIO11 da Raspberry
39 IO_PORT "sclk" IO_TYPE=LVCMS33 PULL_MODE=DOWN;
40 IO_LOC "cs_n" 42; // entrada, vem do GPIO6 da Raspberry
41 IO_PORT "cs_n" IO_TYPE=LVCMS33 PULL_MODE=UP;
42 IO_LOC "miso" 33; // saida para o GPIO20 da Raspberry (banco 2: nivel alto = 2,5 V)
43 IO_PORT "miso" IO_TYPE=LVCMS25 DRIVE=8;
```

Fonte: elaborado pelo autor (2026).

4. SOFTWARE ASSEMBLY ARM 64 BITS

O software da Raspberry Pi é Assembly AArch64 puro, montado com GNU as/ld, **sem libc**: entrada e saída, memória e tempo usam syscalls Linux diretas (openat, mmap, close, write, clock_gettime, nanosleep, exit). Todas as rotinas seguem a convenção **AAPCS64** (argumentos em x0–x7, retorno em x0, x19–x28 preservados), o que permite agrupá-las na biblioteca estática libbembarcado.a.

Tabela 10 – Organização do código ARM

Arquivo	Rotinas	Papel
lib/gpio_lib.s	gpio_map_init, gpio_configura_pino, gpio_escreve, gpio_le	acesso direto aos registradores de GPIO
lib/protocolo_serial_gpio.s	protocolo_serial_configura_pinos, protocolo_serial_le_quadro	lado mestre do link (bit-bang)
lib/telemetria.s	telemetria_valida, telemetria_formata	valida e traduz o quadro em texto
lib/strings_lib.s	uint_to_dec, str_copia, escreve_fd	conversão e escrita de texto
monitor_semaforo.s	_start	laço de leitura e impressão 1×/s

Fonte: elaborado pelo autor (2026).

4.1 Acesso aos GPIOs

gpio_map_init abre /dev/gpiomem e mapeia 4 KiB com os registradores do BCM2710. Se não conseguir, devolve 0 e o programa termina com erro — não há modo simulado. gpio_configura_pino calcula o registrador GPFSELn (pino/10) e o deslocamento (3 bits por pino) com udiv/msub; gpio_escreve grava apenas a máscara do pino em GPSET0 ou GPCLR0 (registradores só de escrita, escolhidos com csel), e gpio_le lê o bit em GPLEV0 (Figura 19).

Figura 19 – Rotinas de GPIO com LDR/STR

```
asm/lib/gpio_lib.s · linhas 73-113

73 // ---- gpio_configura_pino ----
74 // GPFSELn: 10 pinos por registrador, 3 bits por pino, registrador n no
75 // offset 4*n. 000 = entrada, 001 = saida.
76 gpio_configura_pino:
77     mov     w9, #10
78     udiv    w10, w1, w9           // w10 = indice do GPFSELn (pino / 10)
79     msub    w11, w10, w9, w1     // w11 = posicao no registrador (pino % 10)
80     add     w11, w11, w11, lsl #1 // w11 = deslocamento em bits (posicao * 3)
81     add     x10, x0, x10, lsl #2 // x10 = endereco de GPFSELn
82
83     ldr     w3, [x10]
84     mov     w4, #0b111
85     lsl     w4, w4, w11
86     bic     w3, w3, w4           // 000 = entrada
87     cbz     w2, 1f
88     mov     w4, #0b001
89     lsl     w4, w4, w11
90     orr     w3, w3, w4           // 001 = saida
91 1: str     w3, [x10]
92     ret
93
94 // ---- gpio_escreve ----
95 // GPSET0/GPCLR0 sao so' de escrita: grava apenas a mascara do pino, sem
96 // ler antes, e os demais pinos nao mudam.
97 gpio_escreve:
98     mov     w3, #1
99     lsl     w3, w3, w1           // w3 = mascara do pino
100    mov     x4, #GPCLR0_OFF
101    mov     x5, #GPSET0_OFF
102    cmp     w2, #0
103    csel     x4, x4, x5, eq       // nivel 0 → GPCLR0, nivel 1 → GPSET0
104    str     w3, [x0, x4]
105    ret
106
107 // ---- gpio_le ----
108 gpio_le:
109     ldr     w3, [x0, #GPLEV0_OFF]
110     lsr     w3, w3, w1
111     and     w0, w3, #1
112     ret
```

Fonte: elaborado pelo autor (2026).

4.2 Driver do link serial

protocolo_serial_le_quadro (Figura 20) baixa cs_n, e em um laço de 16 voltas lê miso, acumula o bit com orr w23, w0, w23, lsl #1, sobe e desce scl_k com um atraso entre as transições, e no fim sobe cs_n. A leitura acontece **antes** de cada subida de scl_k, como exige o modo 0, e o atraso de dezenas de microssegundos fica muito acima dos ~150 ns que a FPGA precisa para sincronizar.

Figura 20 – Leitura de um quadro por bit-bang

```
asm/lib/protocolo_serial_gpio.s · linhas 61-121

61 // ---- protocolo_serial_le_quadro ----
62 protocolo_serial_le_quadro:
63     stp    x29, x30, [sp, #-64]!
64     mov    x29, sp
65     stp    x19, x20, [sp, #16]
66     stp    x21, x22, [sp, #32]
67     stp    x23, x24, [sp, #48]
68     mov    x19, x0                // base dos registradores de GPIO
69     mov    w20, w1                // pino sclk
70     mov    w21, w2                // pino cs_n
71     mov    w22, w3                // pino miso
72     mov    w23, #0                // quadro sendo montado
73     mov    w24, #16               // bits que faltam receber
74
75     mov    x0, x19
76     mov    w1, w21
77     mov    w2, #0
78     bl     gpio_escreve           // cs_n = 0: início do quadro
79     bl     atraso
80
81     // um bit por volta; termina quando os 16 bits foram lidos
82     .Lproximo_bit:
83     mov    x0, x19
84     mov    w1, w22
85     bl     gpio_le                // w0 = bit atual em miso
86     orr    w23, w0, w23, lsl #1
87
88     mov    x0, x19
89     mov    w1, w20
90     mov    w2, #1
91     bl     gpio_escreve           // sclk = 1: FPGA avança para o próximo bit
92     bl     atraso
93     mov    x0, x19
94     mov    w1, w20
95     mov    w2, #0
96     bl     gpio_escreve           // sclk = 0
97     bl     atraso
98
99     subs   w24, w24, #1
100    b.ne   .Lproximo_bit
101
102    mov    x0, x19
103    mov    w1, w21
104    mov    w2, #1
105    bl     gpio_escreve           // cs_n = 1: fim do quadro
106    bl     atraso
107
108    and    w0, w23, #0xFFFF
109    ldp    x23, x24, [sp, #48]
110    ldp    x21, x22, [sp, #32]
111    ldp    x19, x20, [sp, #16]
112    ldp    x29, x30, [sp], #64
113    ret
114
115 // espera ocupada de ATRASO_ITER voltas (so' usa w9)
116 atraso:
117     mov    w9, #ATRASO_ITER
118 1:     subs   w9, w9, #1
119     b.ne   1b
120     ret
```

Fonte: elaborado pelo autor (2026).

4.3 Validação e decodificação do quadro

telemetria_valida (Figura 21) é um pequeno **parser de bitfields**: extrai cada campo com ubfx e rejeita o quadro se o marcador não for 1010, se a cor ou o fluxo valerem 11, ou se o pedestre estiver verde com o carro fora do vermelho. telemetria_formata (Figura 22) monta

a linha de texto usando **tabelas de ponteiros indexadas** pelos campos (`ldr x1, [x9, w10, uxtw #3]`) e três **macros parametrizadas** (`acrescenta`, `acrescenta_tabela`, `acrescenta_numero`).

Figura 21 – Validação do quadro com extração de bitfields



```
asm/lib/telemetria.s · linhas 50-68

50 // ---- telemetria_valida ----
51 telemetria_valida:
52     ubfx     w1, w0, #12, #4
53     cmp      w1, #MARCADOR
54     b.ne     .Lvalida_nao
55     ubfx     w1, w0, #10, #2      // w1 = cor dos carros
56     cmp      w1, #3
57     b.eq     .Lvalida_nao
58     ubfx     w2, w0, #6, #2      // w2 = fluxo
59     cmp      w2, #3
60     b.eq     .Lvalida_nao
61     tbz      w0, #9, .Lvalida_sim // pedestre vermelho: combina com qualquer cor
62     cbnz     w1, .Lvalida_nao    // pedestre verde exige carro vermelho (00)
63 .Lvalida_sim:
64     mov      w0, #1
65     ret
66 .Lvalida_nao:
67     mov      w0, #0
68     ret
```

Fonte: elaborado pelo autor (2026).

Figura 22 – Formatação com tabelas indexadas e macros

```
asm/lib/telemetria.s · linhas 97-142

97 // ---- telemetria_formata ----
98 // Linha valida:  carros=VERDE pedestres=VERMELHO restante=4s fluxo=BAIXO pedido=SIM
99 // Linha invalida: QUADRO INVALIDO (valor=N): confira fiacao, GND comum e se a FPGA esta gravada
100 telemetria_formata:
101     stp     x29, x30, [sp, #-48]!
102     mov     x29, sp
103     stp     x19, x20, [sp, #16]
104     str     x21, [sp, #32]
105     and     w19, w0, #0xFFFF           // quadro recebido
106     mov     x20, x1                     // inicio do destino
107     mov     x21, x1                     // posicao de escrita no destino
108
109     bl      telemetria_valida
110     cbz     w0, .Lformata_invalido
111
112     acrescenta txt_carros
113     ubfx    w10, w19, #10, #2
114     acrescenta_tabela tab_cor_carro, w10
115     acrescenta txt_pedestres
116     ubfx    w10, w19, #9, #1
117     acrescenta_tabela tab_pedestre, w10
118     acrescenta txt_restante
119     and     x10, x19, #0x3F
120     acrescenta_numero x10
121     acrescenta txt_segundos
122     acrescenta txt_fluxo
123     ubfx    w10, w19, #6, #2
124     acrescenta_tabela tab_fluxo, w10
125     acrescenta txt_pedido
126     ubfx    w10, w19, #8, #1
127     acrescenta_tabela tab_pedido, w10
128     b       .Lformata_fim
129
130 .Lformata_invalido:
131     acrescenta txt_invalido
132     acrescenta_numero x19
133     acrescenta txt_dica
134
135 .Lformata_fim:
136     acrescenta txt_fim_linha
137     sub     x0, x21, x20               // caracteres escritos
138     ldr     x21, [sp, #32]
139     ldp     x19, x20, [sp, #16]
140     ldp     x29, x30, [sp], #48
141     ret
```

Fonte: elaborado pelo autor (2026).

4.4 Programa monitor

O `monitor_semaforo.s` (Figura 23) configura os pinos e entra num laço: lê um quadro, calcula os segundos desde o início com `clock_gettime(CLOCK_MONOTONIC)`, monta `[t=Ns]` seguido da linha decodificada, escreve com `write` e dorme 1 s com `nanosleep`. Uma linha típica é `[t=12s] carros=VERDE pedestres=VERMELHO restante=4s fluxo=MEDIO pedido=SIM`.

Figura 23 – Laço principal do monitor

```
asm/monitor_semaforo.s · linhas 66-115

66      // uma leitura por volta; so' termina com Ctrl+C
67      .Lmonitora:
68      mov     x0, x19
69      mov     w1, #PIN0_SCLK
70      mov     w2, #PIN0_CS_N
71      mov     w3, #PIN0_MISO
72      bl      protocolo_serial_le_quadro
73      mov     w21, w0          // w21 = quadro recebido
74
75      segundos_monotonicos x22
76      sub     x22, x22, x20    // x22 = segundos desde o inicio
77
78      // monta "[t=<s>s] " + estado decodificado em "linha"
79      adrp    x23, linha
80      add     x23, x23, :lo12:linha    // x23 = posicao de escrita na linha
81      mov     x0, x23
82      adrp    x1, txt_prefixo
83      add     x1, x1, :lo12:txt_prefixo
84      bl      str_copia
85      mov     x23, x0
86      mov     x0, x22
87      mov     x1, x23
88      bl      uint_to_dec
89      add     x23, x23, x0
90      mov     x0, x23
91      adrp    x1, txt_sufixo
92      add     x1, x1, :lo12:txt_sufixo
93      bl      str_copia
94      mov     x23, x0
95      mov     w0, w21
96      mov     x1, x23
97      bl      telemetria_formata
98      add     x23, x23, x0
99
100     mov     x0, #1
101     adrp    x1, linha
102     add     x1, x1, :lo12:linha
103     sub     x2, x23, x1
104     bl      escreve_fd
105
106     // espera 1 segundo
107     adrp    x0, tempo
108     add     x0, x0, :lo12:tempo
109     mov     x1, #1
110     stp     x1, xzr, [x0]          // tv_sec = 1, tv_nsec = 0
111     mov     x1, #0
112     mov     x8, #101              // nanosleep
113     svc     #0
114     b       .Lmonitora
115
```

Fonte: elaborado pelo autor (2026).

5. TESTES E VALIDAÇÃO

5.1 Metodologia

Cada módulo tem um testbench **auto-verificável** em tb/: o próprio testbench compara os sinais com o valor esperado e imprime [OK] ou [FALHA] por caso e um RESULTADO final. Os testes rodam com `make sim` (Icarus Verilog, -wall), cada um gera um arquivo VCD em build/, e cada VCD foi aberto no **GTKWave** para análise das formas de onda. Os testbenches reduzem os tempos (debounce de 2–4 ciclos, “segundo” de 1 a 40 ciclos) para simular em microssegundos o que na placa dura segundos; a lógica é a mesma. As capturas a seguir são da tela do GTKWave com os sinais relevantes de cada teste.

Tabela 11 – Resumo dos testbenches

Testbench	Verificações	O que comprova	Resultado
tb_debounce	4	repique ignorado; níveis estáveis propagam após 2 + N ciclos	passou
tb_sensor_veiculo	2	1 pulso por veículo, mesmo com repique	passou
tb_botao_pedestre	3	toque curto ignorado; latch; limpeza	passou
tb_contador_tempo	6	carga, decremento, parada em zero, prioridade	passou
tb_estado_basico_decoder	4	4 códigos de cor; inválido = vermelho	passou
tb_bram_historico	7	ponteiro circular e leitura síncrona	passou
tb_media_movel_dsp	14	BRAM + DSP contra modelo de referência	passou
tb_protocolo_serial	3	quadro de 16 bits e congelamento	passou
tb_fsm_semaforo	8	ciclo completo, regra do pedido novo, segurança	passou
tb_top_semaforo	11	sistema inteiro só pelos pinos	passou
Total	62		10/10

Fonte: elaborado pelo autor (2026).

5.2 Debounce

Figura 24 – Saída de `make sim-debounce`

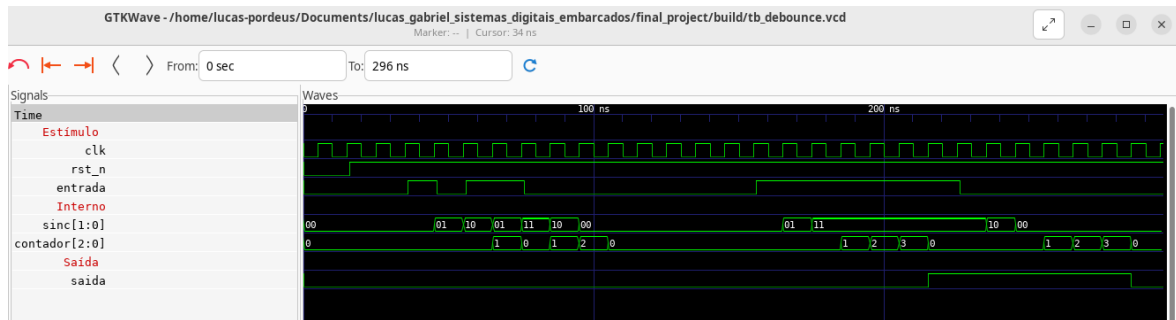


```
terminal · make sim-debounce

$ make sim-debounce
== tb_debounce ==
iverilog -g2012 -Wall -Wno-timescale -o build/tb_debounce.vvp -s tb_debounce tb/tb_debounce.v rtl/debounce.v rtl/sensor_veiculo.v rtl/botao_pedestre.v rtl/prescaler.v rtl/contador_tempo.v rtl/fsm_semaforo.v rtl/estado_basico_decoder.v rtl/bram_historico.v rtl/media_movel_dsp.v rtl/protocolo_serial.v rtl/top_semaforo.v
vvp -n build/tb_debounce.vvp
VCD info: dumpfile build/tb_debounce.vcd opened for output.
[OK] repique ignorado
[OK] saida ainda nao muda antes de N + 2 ciclos
[OK] nivel 1 estavel chega a saida
[OK] nivel 0 estavel chega a saida
RESULTADO: TODOS OS CASOS PASSARAM
tb/tb_debounce.v:52: $finish called at 296000 (1ps)
```

Fonte: elaborado pelo autor (2026).

Figura 25 – GTKWave – tb_debounce: repique e níveis estáveis



Fonte: elaborado pelo autor (2026).

Análise da forma de onda:

- Entre 36 e 76 ns a entrada oscila (dois repiques). O contador começa a contar a cada mudança (1, 0, 1, 2, 0), mas é zerado antes de chegar a $N = 4$, e a saída permanece em 0: o repique foi descartado.
- Em 156 ns a entrada fica estável em 1. Vê-se `sinc[1:0]` passar por 1 e 3 (os dois flip-flops de sincronização), o contador subir 1, 2, 3, e a saída ir a 1 em 215 ns — 6 ciclos depois (2 de sincronização + 4 de estabilidade).
- A volta para 0 em 226 ns aparece na saída em 285 ns pelo mesmo caminho. A latência é fixa e previsível: no hardware, com $N = 540\,000$, ela vale 20 ms, acima do repique típico de um botão (1 a 10 ms).

5.3 Sensor de veículo

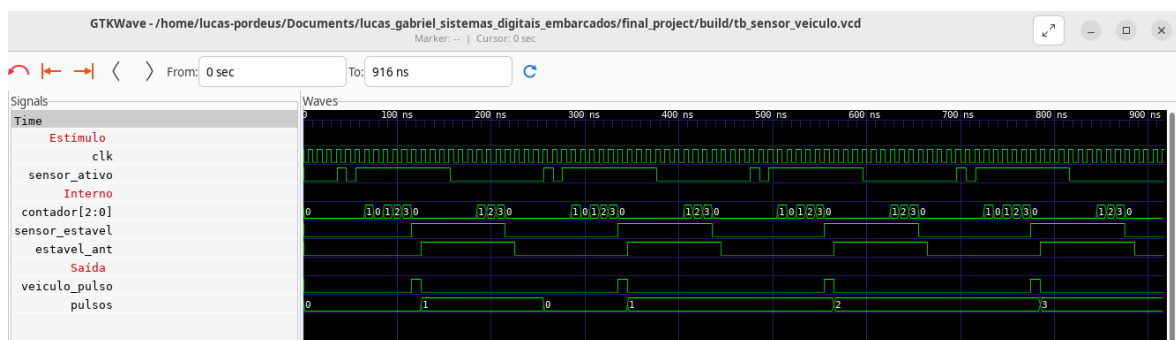
Figura 26 – Saída de make sim-sensor_veiculo

```
terminal · make sim-sensor_veiculo

$ make sim-sensor_veiculo
===== tb_sensor_veiculo =====
iverilog -g2012 -Wall -Wno-timescale -o build/tb_sensor_veiculo.vvp -s tb_sensor_veiculo tb/tb_sensor_veiculo.v rtl/debounce.v rtl/sensor_veiculo.v rtl/botao_pedestre.v rtl/prescaler.v rtl/contador_tempo.v rtl/fsm_semaforo.v rtl/estado_basico_decoder.v rtl/bram_historico.v rtl/media_movel_dsp.v rtl/protocolo_serial.v rtl/top_semaforo.v
vvp -n build/tb_sensor_veiculo.vvp
VCD info: dumpfile build/tb_sensor_veiculo.vcd opened for output.
[OK] 1 veículo com repique = 1 pulso (1 pulsos)
[OK] 3 veículos = 3 pulsos (3 pulsos)
RESULTADO: TODOS OS CASOS PASSARAM
tb/tb_sensor_veiculo.v:56: $finish called at 916000 (1ps)
```

Fonte: elaborado pelo autor (2026).

Figura 27 – GTKWave – tb_sensor_veiculo: um pulso por veículo



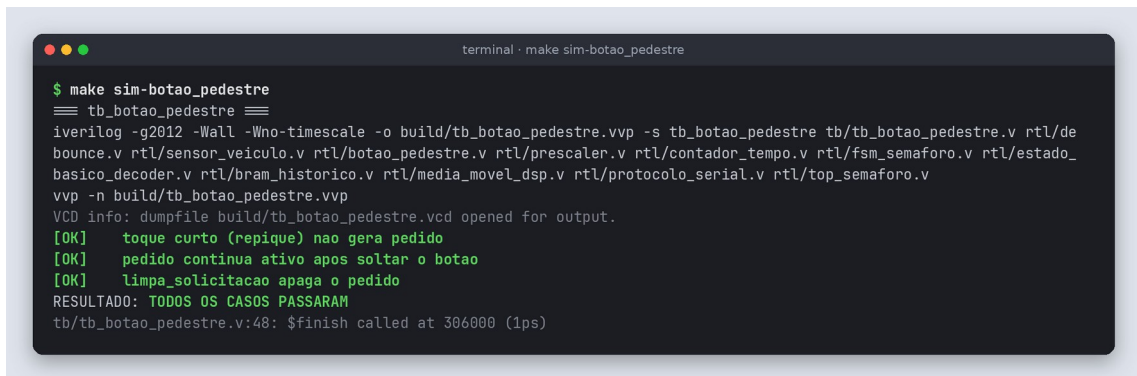
Fonte: elaborado pelo autor (2026).

Análise da forma de onda:

- Cada passagem começa com um repique de 10 ns em sensor_ativo; o contador do debounce reinicia e o sinal filtrado sensor_estavel só sobe depois da estabilização (115, 335, 555 e 775 ns).
- veiculo_pulso é exatamente sensor_estavel & ~estavel_ant: dura um único ciclo (10 ns) na subida, embora o veículo fique 100 ns na frente do sensor.
- O contador pulsos do testbench mostra 1 após o primeiro veículo, é zerado, e chega a 3 após os três seguintes — nenhuma contagem extra apesar dos repiques.

5.4 Botão de pedestre

Figura 28 – Saída de make sim-botao_pedestre

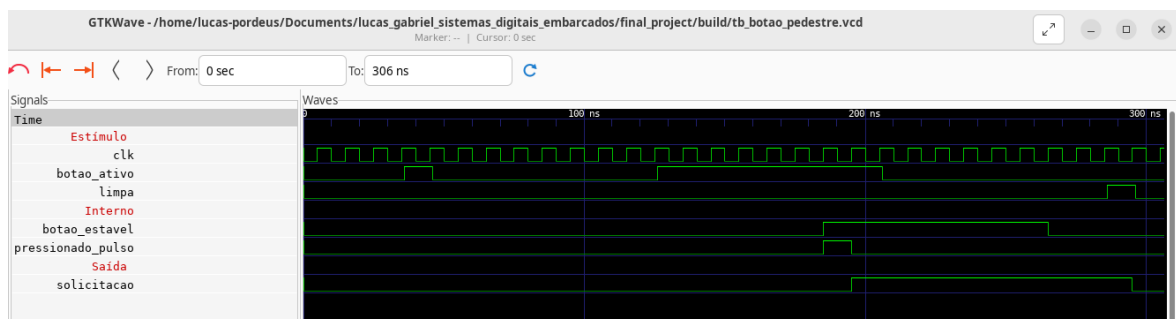


```
terminal - make sim-botao_pedestre

$ make sim-botao_pedestre
== tb_botao_pedestre ==
iverilog -g2012 -Wall -Wno-timescale -o build/tb_botao_pedestre.vvp -s tb_botao_pedestre tb/tb_botao_pedestre.v rtl/debounce.v rtl/sensor_veiculo.v rtl/botao_pedestre.v rtl/prescaler.v rtl/contador_tempo.v rtl/fsm_semaforo.v rtl/estado_basico_decoder.v rtl/bram_historico.v rtl/media_movel_dsp.v rtl/protocolo_serial.v rtl/top_semaforo.v
vvp -n build/tb_botao_pedestre.vvp
VCD info: dumpfile build/tb_botao_pedestre.vcd opened for output.
[OK] toque curto (repique) nao gera pedido
[OK] pedido continua ativo apos soltar o botao
[OK] limpa_solicitacao apaga o pedido
RESULTADO: TODOS OS CASOS PASSARAM
tb/tb_botao_pedestre.v:48: $finish called at 306000 (1ps)
```

Fonte: elaborado pelo autor (2026).

Figura 29 – GTKWave – tb_botao_pedestre: latch do pedido



Fonte: elaborado pelo autor (2026).

Análise da forma de onda:

- O toque curto de 36–46 ns não chega a botao_estavel e não gera pedido.
- No aperto real (126–206 ns), botao_estavel sobe em 185 ns, pressionado_pulso dura um ciclo e solicitacao sobe em 195 ns.
- O botão é solto em 206 ns e botao_estavel cai em 265 ns, mas solicitacao **continua em 1**: o pedido foi travado. Só o pulso de limpa (286 ns) o apaga em 295 ns.

5.5 Contador de tempo

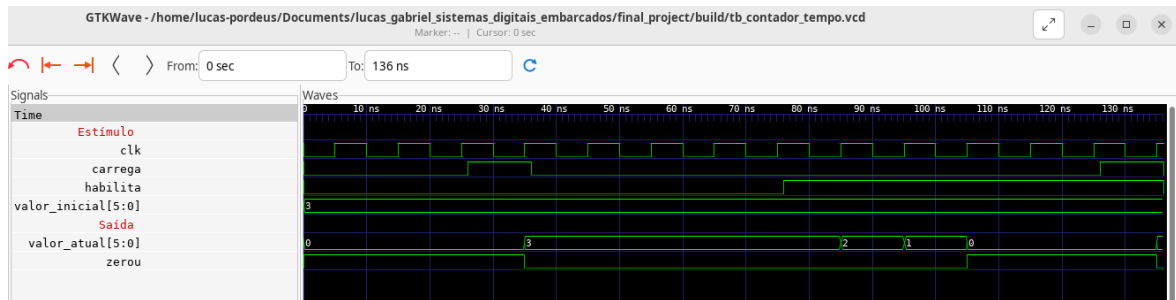
Figura 30 – Saída de make sim-contador_tempo

```
terminal · make sim-contador_tempo

$ make sim-contador_tempo
== tb_contador_tempo ==
iverilog -g2012 -Wall -Wno-timescale -o build/tb_contador_tempo.vvp -s tb_contador_tempo tb/tb_contador_tempo.v rtl/debounce.v rtl/sensor_veiculo.v rtl/botao_pedestre.v rtl/prescaler.v rtl/contador_tempo.v rtl/fsm_semaforo.v rtl/estado_basico_decoder.v rtl/bram_historico.v rtl/media_movel_dsp.v rtl/protocolo_serial.v rtl/top_semaforo.v
vvp -n build/tb_contador_tempo.vvp
VCD info: dumpfile build/tb_contador_tempo.vcd opened for output.
[OK] zerado apos reset (valor=0)
[OK] carrega valor_inicial (valor=3)
[OK] sem habilita, nao decrementa (valor=3)
[OK] decrementa 1 por ciclo habilitado (valor=1)
[OK] para em zero (valor=0)
[OK] carrega tem prioridade sobre habilita (valor=3)
RESULTADO: TODOS OS CASOS PASSARAM
tb/tb_contador_tempo.v:54: $finish called at 136000 (1ps)
```

Fonte: elaborado pelo autor (2026).

Figura 31 – GTKWave – tb_contador_tempo: carga, decremento e parada



Fonte: elaborado pelo autor (2026).

Análise da forma de onda:

- carrega em 26 ns leva valor_atual a 3 em 35 ns e zerou cai.
- Sem habilita o valor fica parado; com habilita (a partir de 76 ns) desconta 1 por ciclo: 2, 1, 0 (85–105 ns).
- Em 0, zerou sobe e o valor não passa para 63 (não há estouro). Em 126 ns carrega e habilita estão ativos ao mesmo tempo e o valor volta a 3: a carga tem prioridade.

5.6 Decodificador de LEDs

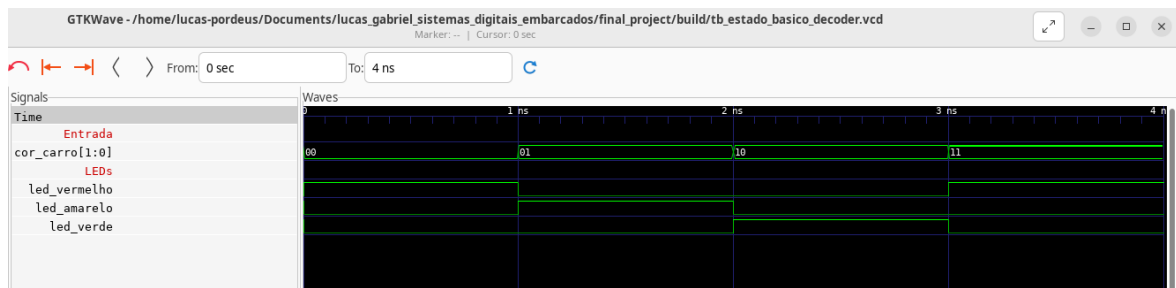
Figura 32 – Saída de make sim-estado_basico_decoder

```
terminal · make sim-estado_basico_decoder

$ make sim-estado_basico_decoder
== tb_estado_basico_decoder ==
iverilog -g2012 -Wall -Wno-timescale -o build/tb_estado_basico_decoder.vvp -s tb_estado_basico_decoder tb/tb_estado_basico_decoder.v rtl/debounce.v rtl/sensor_veiculo.v rtl/botao_pedestre.v rtl/prescaler.v rtl/contador_tempo.v rtl/fsm_semaforo.v rtl/estado_basico_decoder.v rtl/bram_historico.v rtl/media_movel_dsp.v rtl/protocolo_serial.v rtl/top_semaforo.v
vvp -n build/tb_estado_basico_decoder.vvp
VCD info: dumpfile build/tb_estado_basico_decoder.vcd opened for output.
[OK] cor_carro=00 → LEDs (V,A,Vd)=100
[OK] cor_carro=01 → LEDs (V,A,Vd)=010
[OK] cor_carro=10 → LEDs (V,A,Vd)=001
[OK] cor_carro=11 → LEDs (V,A,Vd)=100
RESULTADO: TODOS OS CASOS PASSARAM
tb/tb_estado_basico_decoder.v:34: $finish called at 4000 (1ps)
```

Fonte: elaborado pelo autor (2026).

Figura 33 – GTKWave – tb_estado_basico_decoder: cores dos carros



Fonte: elaborado pelo autor (2026).

Análise da forma de onda:

- Para `cor_carro = 00, 01 e 10` acende exatamente um LED: vermelho, amarelo e verde.
- O código inválido `11` acende o **vermelho**, que é a condição segura. Por ser combinacional, as saídas mudam no mesmo instante da entrada.

5.7 Histórico em BRAM

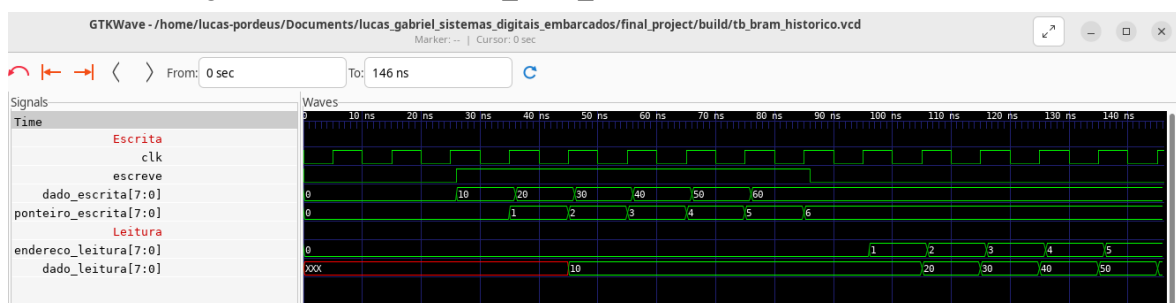
Figura 34 – Saída de `make sim-bram_historico`

```
terminal - make sim-bram_historico

$ make sim-bram_historico
== tb_bram_historico ==
iverilog -g2012 -Wall -Wno-timescale -o build/tb_bram_historico.vvp -s tb_bram_historico tb/tb_bram_historico.v rtl/de
bounce.v rtl/sensor_veiculo.v rtl/botao_pedestre.v rtl/prescaler.v rtl/contador_tempo.v rtl/fsm_semaforo.v rtl/estado_
basico_decoder.v rtl/bram_historico.v rtl/media_movel_dsp.v rtl/protocolo_serial.v rtl/top_semaforo.v
vvp -n build/tb_bram_historico.vvp
VCD info: dumpfile build/tb_bram_historico.vcd opened for output.
[OK] ponteiro avancou para 6
[OK] memoria[0]=10
[OK] memoria[1]=20
[OK] memoria[2]=30
[OK] memoria[3]=40
[OK] memoria[4]=50
[OK] memoria[5]=60
RESULTADO: TODOS OS CASOS PASSARAM
tb/tb_bram_historico.v:53: $finish called at 146000 (1ps)
```

Fonte: elaborado pelo autor (2026).

Figura 35 – GTKWave – tb_bram_historico: escrita e leitura síncrona



Fonte: elaborado pelo autor (2026).

Análise da forma de onda:

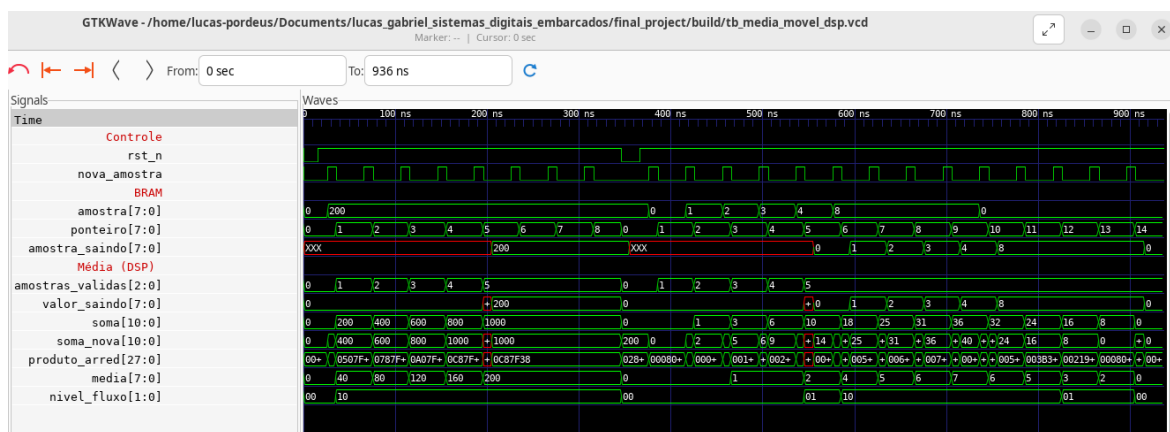
- Com `escreve` ativo de 26 a 86 ns, os valores 10 a 60 são gravados e o `ponteiro_escrita` avança 1 a cada borda até 6.

- ## 5.8 Média móvel com BRAM e DSP

```
terminal · make sim-media_movel_dsp

$ make sim-media_movel_dsp
== tb_media_movel_dsp ==
iverilog -g2012 -Wall -Wno-timescale -o build/tb_media_movel_dsp.vvp -s tb_media_movel_dsp tb/tb_media_movel_dsp.v rtl
/debounce.v rtl/sensor_veiculo.v rtl/botao_pedestre.v rtl/prescaler.v rtl/contador_tempo.v rtl/fsm semaforo.v rtl/esta
do_basico_decoder.v rtl/bram_historico.v rtl/media_movel_dsp.v rtl/protocolo_serial.v rtl/top_semaforo.v
vvp -n build/tb_media_movel_dsp.vvp
VCD info: dumpfile build/tb_media_movel_dsp.vcd opened for output.
[OK] amostra 0=0: media=0 nivel=0
[OK] amostra 1=1: media=0 nivel=0
[OK] amostra 2=2: media=1 nivel=0
[OK] amostra 3=3: media=1 nivel=0
[OK] amostra 4=4: media=2 nivel=1
[OK] amostra 5=8: media=4 nivel=2
[OK] amostra 6=8: media=5 nivel=2
[OK] amostra 7=8: media=6 nivel=2
[OK] amostra 8=8: media=7 nivel=2
[OK] amostra 9=0: media=6 nivel=2
[OK] amostra 10=0: media=5 nivel=2
[OK] amostra 11=0: media=3 nivel=1
[OK] amostra 12=0: media=2 nivel=1
[OK] amostra 13=0: media=0 nivel=0
RESULTADO: TODOS OS CASOS PASSARAM
tb/tb_media_movel_dsp.v:84: $finish called at 936000 (1ps)
```

Figura 37 – GTKWave – tb_media_movel_dsp: BRAM ligada à média, como no topo



Análise da forma de onda:

- **Antes do reset (0–346 ns):** o testbench grava oito amostras de valor 200. A media sobe 40, 80, 120, 160 e 200 enquanto a janela enche, e depois fica em 200: a amostra que sai (200, lida da BRAM em 205 ns) é igual à que entra.
- **Reset em 346 ns:** ponteiro, soma, media e amostras_validas voltam a 0, mas a BRAM mantém os 200. Na leitura de ponteiro - 5 aparece xxx (posição nunca gravada). Ainda assim valor_saindo fica em 0 enquanto amostras_validas < 5 — o

lixo nunca entra na soma. É a demonstração de que dispensar o reset da memória é seguro.

- **Sequência 0, 1, 2, 3, 4, 8, 8, 8, 8, 0, 0, 0, 0, 0:** a soma evolui 0, 1, 3, 6, 10, 18, 25, 31, 36, 32, 24, 16, 8, 0, e a media 0, 0, 1, 1, 2, 4, 5, 6, 7, 6, 5, 3, 2, 0. A partir da 6ª amostra amostra_saindo traz os valores corretos da BRAM (0, 1, 2, 3, 4, 8).
- produto_arred mostra o resultado da multiplicação Q16 (para soma 25 vale 0x0057FFB, cujos bits [23:16] dão 5), e nivel_fluxo passa de baixo (00) para médio (01) em 545 ns, alto (10) em 585 ns, e volta a médio e baixo quando as amostras zeram.
- As 14 médias e níveis coincidem com o modelo de referência round(soma/5) do testbench.

5.9 Protocolo serial

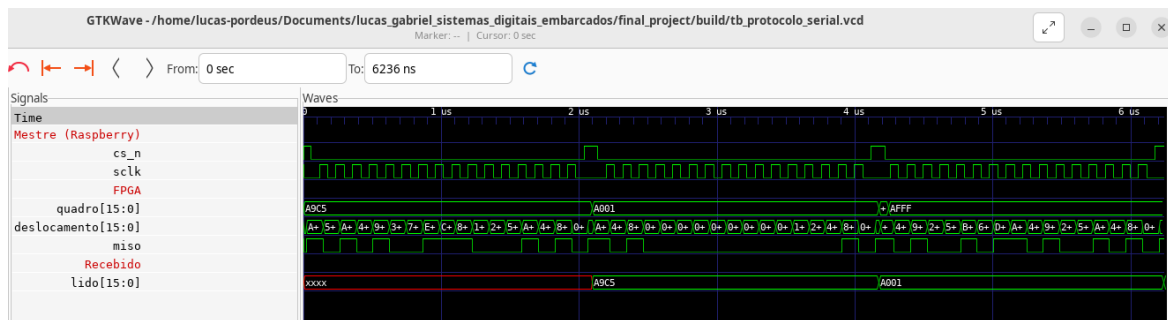
Figura 38 – Saída de make sim-protocolo_serial

```
terminal - make sim-protocolo_serial

$ make sim-protocolo_serial
== tb_protocolo_serial ==
iverilog -g2012 -Wall -Wno-timescale -o build/tb_protocolo_serial.vvp -s tb_protocolo_serial tb/tb_protocolo_serial.v
rtl/debounce.v rtl/sensor_veiculo.v rtl/botao_pedestre.v rtl/prescaler.v rtl/contador_tempo.v rtl/fsm_semaforo.v rtl/e
stado_basico_decoder.v rtl/bram_historico.v rtl/media_movel_dsp.v rtl/protocolo_serial.v rtl/top_semaforo.v
vvp -n build/tb_protocolo_serial.vvp
VCD info: dumpfile build/tb_protocolo_serial.vcd opened for output.
[OK] quadro lido bit a bit (a9c5)
[OK] quadro atualizado entre transferencias (a001)
[OK] quadro congelado durante a transferencia (a5a5)
RESULTADO: TODOS OS CASOS PASSARAM
tb/tb_protocolo_serial.v:70: $finish called at 6236000 (1ps)
```

Fonte: elaborado pelo autor (2026).

Figura 39 – GTKWave – tb_protocolo_serial: três quadros completos



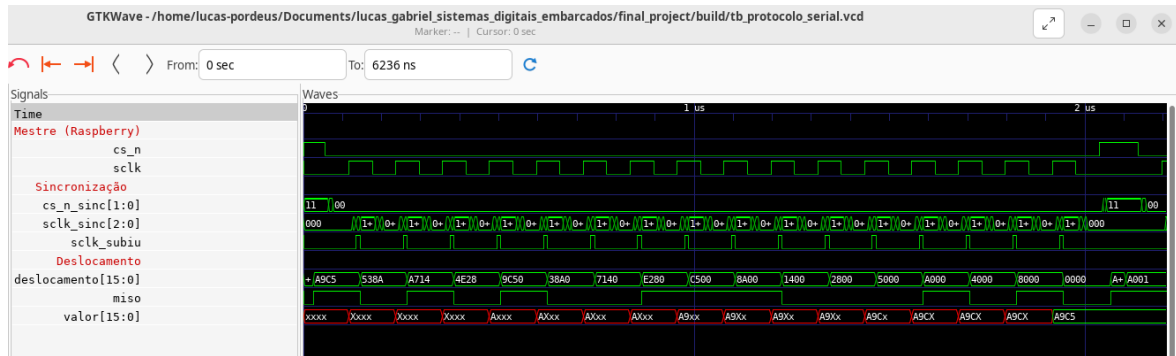
Fonte: elaborado pelo autor (2026).

Análise da forma de onda:

- São três transferências delimitadas por cs_n baixo, cada uma com 16 pulsos de sclk.
- O registro deslocamento começa com o quadro e é deslocado a cada subida de sclk; miso é sempre o bit 15. O valor lido pelo mestre do testbench é A9C5 e depois A001, iguais aos quadros enviados.
- No terceiro quadro o testbench muda quadro para AFFF no meio da transferência (4,25 µs), mas o valor lido continua A5A5: o quadro é congelado quando cs_n desce.

5.10 Detalhe de um quadro serial

Figura 40 – GTKWave – tb_protocolo_serial: zoom no primeiro quadro (0 a 2,2 µs)



Fonte: elaborado pelo autor (2026).

Análise da forma de onda:

- cs_n_sync e sclk_sync mostram o sinal externo atravessando os flip-flops de sincronização; sclk_subiu é o pulso de um ciclo na borda de subida já sincronizada.
- deslocamento vale A9C5 e passa a 538A, A714, 4E28 ... a cada sclk_subiu (deslocamento de 1 bit à esquerda).
- Em valor[15:0] do testbench vê-se o quadro sendo montado bit a bit (AXXX, A9XX, A9CX, ...) até ficar A9C5 no 16º bit: a leitura antes da subida de sclk recupera o quadro corretamente.

5.11 Máquina de estados

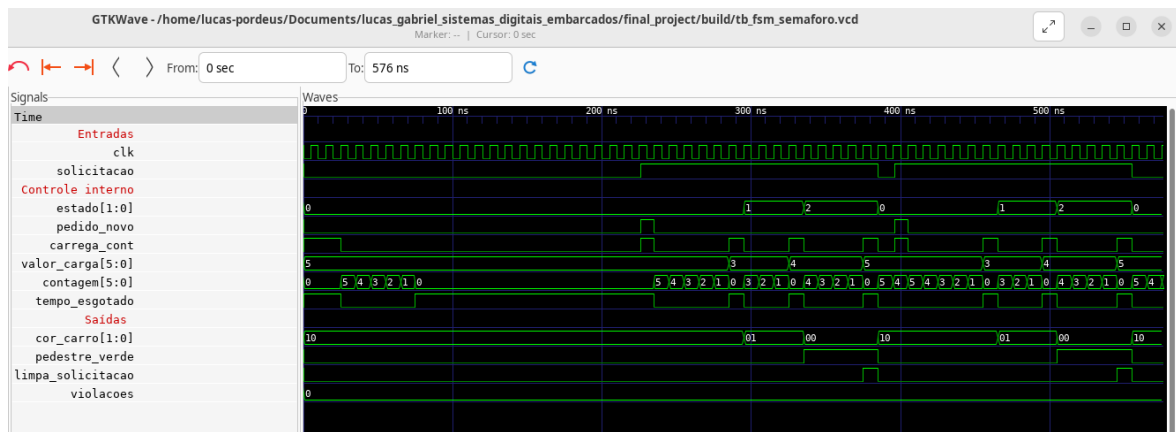
Figura 41 – Saída de make sim-fsm_semaforo

```
terminal · make sim-fsm_semaforo

$ make sim-fsm_semaforo
# tb_fsm_semaforo #
iverilog -g2012 -Wall -Wno-timescale -o build/tb_fsm_semaforo.vvp -s tb_fsm_semaforo tb/tb_fsm_semaforo.v rtl/debounce
.v rtl/sensor_veiculo.v rtl/botao_pedestre.v rtl/prescaler.v rtl/contador_tempo.v rtl/fsm_semaforo.v rtl/estado_basico
_decoder.v rtl/bram_historico.v rtl/media_movel_dsp.v rtl/protocolo_serial.v rtl/top_semaforo.v
vvp -n build/tb_fsm_semaforo.vvp
VCD info: dumpfile build/tb_fsm_semaforo.vcd opened for output.
[OK]   inicia com carros verdes e pedestre vermelho
[OK]   sem pedido, carros continuam verdes
[OK]   pedido tardio: carros ficam mais tempo_verde em verde
[OK]   amarelo dura tempo_amarelo e abre para o pedestre
[OK]   travessia dura tempo_pedestre e volta ao verde
[OK]   pedido apagado ao fim da travessia
[OK]   pedido no inicio do verde nao soma esperas
[OK]   nunca pedestre verde com carro fora do vermelho
RESULTADO: TODOS OS CASOS PASSARAM
tb/tb_fsm_semaforo.v:101: $finish called at 576000 (1ps)
```

Fonte: elaborado pelo autor (2026).

Figura 42 – GTKWave – tb_fsm_semaforo: ciclo completo e regra do pedido novo



Fonte: elaborado pelo autor (2026).

Análise da forma de onda:

- No reset o contador recebe 5 (iniciou) e conta até 0 em 75 ns; sem pedido, estado permanece 0 (CARRO_VERDE) e cor_carro = 10 (verde).
- Em 226 ns chega o pedido: pedido_novo e carrega_cont pulsam juntos e contagem volta a 5 — a espera foi recarregada, embora o verde já estivesse esgotado. Só quando a contagem zera de novo o estado passa a 1 (amarelo, 295 ns).
- valor_carga mostra a duração da fase **que vai começar**: 3 para o amarelo, 4 para a travessia e 5 para o verde. Em 335 ns o estado vai a 2: cor_carro = 00 (vermelho) e pedestre_verde = 1. Em 375 ns limpa_solicitacao pulsa e o pedido é apagado.
- O segundo pedido chega 1 ciclo depois do verde começar (396 ns): o contador, que estava em 4, é recarregado para 5 — a espera não soma com o verde mínimo.
- O sinal violacoes, incrementado sempre que o pedestre está verde com o carro fora do vermelho, permanece **0** do início ao fim.

5.12 Sistema completo (integração)

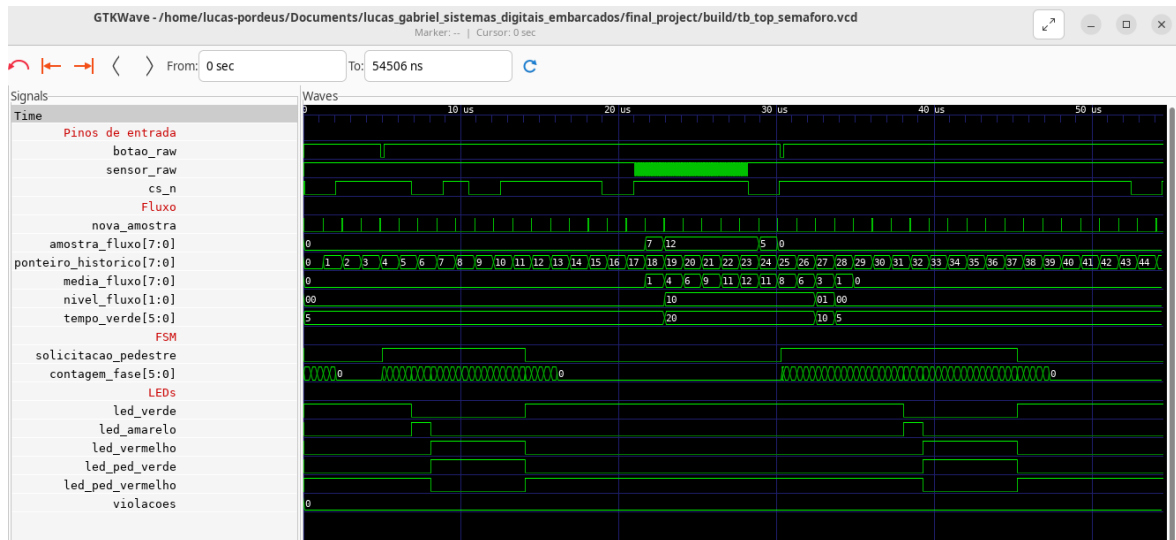
Figura 43 – Saída de make sim-top_semaforo

```
terminal · make sim-top_semaforo

$ make sim-top_semaforo
== tb_top_semaforo ==
iverilog -g2012 -Wall -Wno-timescale -o build/tb_top_semaforo.vvp -s tb_top_semaforo tb/tb_top_semaforo.v rtl/debounce
.v rtl/sensor_veiculo.v rtl/botao_pedestre.v rtl/prescaler.v rtl/contador_tempo.v rtl/fsm_semaforo.v rtl/estado_basico
_decoder.v rtl/bram_historico.v rtl/media_movel_dsp.v rtl/protocolo_serial.v rtl/top_semaforo.v
vvp -n build/tb_top_semaforo.vvp
VCD info: dumpfile build/tb_top_semaforo.vcd opened for output.
[OK] inicio: carros verdes, pedestre vermelho
[OK] telemetria inicial: VERDE / VERMELHO / sem pedido / fluxo baixo
[INFO] fluxo baixo: 4 s do aperto ao amarelo
[OK] fluxo baixo: pedestre espera ~TEMPO_VERDE_BAIXO
[OK] telemetria no amarelo: AMARELO / VERMELHO / pedido pendente
[OK] telemetria na travessia: VERMELHO / pedestre VERDE
[OK] volta ao verde com o pedido apagado
[OK] sensor real eleva o fluxo para ALTO
[INFO] fluxo alto: 19 s do aperto ao amarelo
[OK] fluxo alto: pedestre espera ~TEMPO_VERDE_ALTO
[OK] fluxo alto atrasa o pedestre mais que fluxo baixo
[OK] sem veiculos o fluxo volta a BAIXO
[OK] LEDs sempre coerentes e seguros
RESULTADO: TODOS OS CASOS PASSARAM
tb/tb_top_semaforo.v:151: $finish called at 54506000 (1ps)
```

Fonte: elaborado pelo autor (2026).

Figura 44 – GTKWave – tb_top_semaforo: visão geral (0 a 54,5 μ s; 1 s = 0,4 μ s)



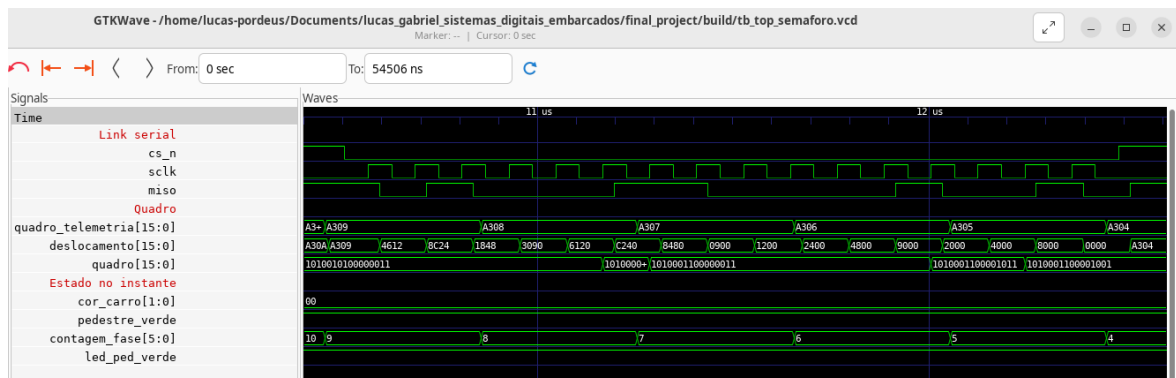
Fonte: elaborado pelo autor (2026).

Análise da forma de onda:

- O teste usa só os pinos reais. Em 4,99 μ s solicitacao_pedestre sobe e contagem_fase é recarregada para 5. O amarelo começa em 6,87 μ s ($\approx$ 4,7 s simulados), dura 3 s e o pedestre fica verde de 8,07 a 14,07 μ s (15 s); solicitacao_pedestre cai ao fim da travessia.
- Entre 20 e 28 μ s o testbench passa veículos no sensor_raw (faixa sólida). As janelas fecham com 7 e 12 carros, a media_fluxo vai a 4 em 22,87 μ s, nivel_fluxo vira 10 (alto) e tempo_verde passa de 5 para 20; o ponteiro_historico avança uma posição a cada janela.
- O segundo pedido (30,30 μ s) é atendido com o amarelo em 38,07 μ s: $\approx$ 19,4 s de espera contra $\approx$ 4,7 s no fluxo baixo. Depois que os veículos param a média cai (12, 11, 8, 6, 3, 1, 0) e o tempo de verde volta a 10 e a 5 — a espera já em curso não muda, porque foi definida no aperto.
- Os LEDs dos carros nunca acendem dois ao mesmo tempo e o pedestre só fica verde com o vermelho dos carros; violacoes permanece 0.

5.13 Telemetria lida pelo link serial

Figura 45 – GTKWave – tb_top_semaforo: zoom numa leitura durante a travessia (10,4 a 12,6 μ s)



Fonte: elaborado pelo autor (2026).

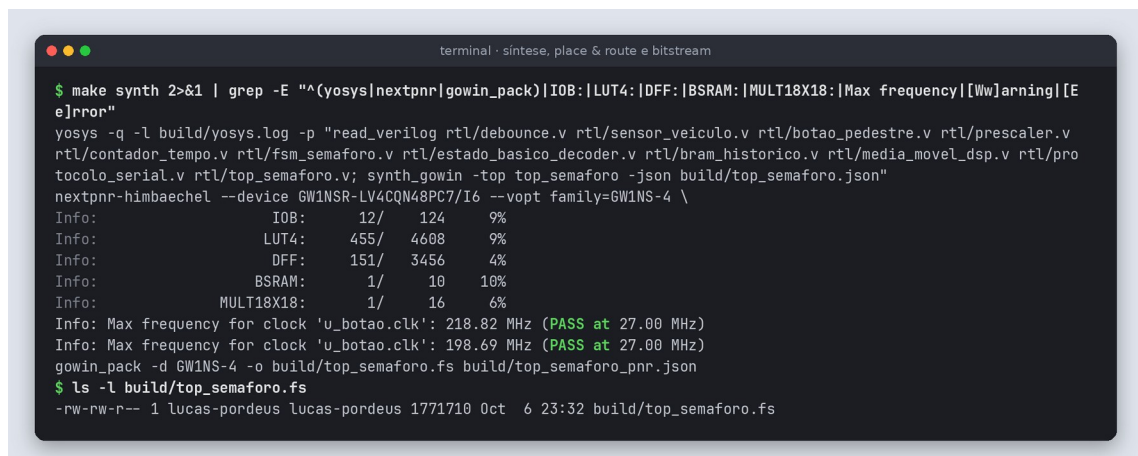
Análise da forma de onda:

- Durante a travessia o quadro_telemetria vale A309, A308, A307 ... : marcador 1010, carros 00 (vermelho), pedestre 1 (verde), pedido 1, fluxo 00 e a contagem regressiva 9, 8, 7 ...
- Quando cs_n desce (10,5 μ s) o deslocamento congela A309 e desloca a cada sclk, embora quadro_telemetria continue mudando para A308 e A307 durante a leitura.
- O registro quadro do testbench é preenchido bit a bit e termina em 1010 00 1 1 00 001001 = A309, exatamente o quadro do instante em que cs_n desceu.

5.14 Síntese, place & route e bitstream

O projeto foi sintetizado com o fluxo aberto (Yosys synth_gowin, nextpnr-himbaechel e Apicula) para o dispositivo GW1NSR-LV4CQN48PC7/I6, com restrição de 27 MHz. Síntese e place & route terminaram **sem nenhum warning**, e o bitstream top_semaforo.fs foi gerado, como mostra a figura a seguir. O mesmo RTL e as mesmas constraints são usados no Gowin EDA.

Figura 46 – Síntese, place & route e geração do bitstream



```
terminal · síntese, place & route e bitstream

$ make synth 2>&1 | grep -E "^(yosys|nextpnr|gowin_pack)|IOB:|LUT4:|DFF:|BSRAM:|MULT18X18:|Max frequency|[Ww]arning|[Ee]rror"
yosys -q -l build/yosys.log -p "read_verilog rtl/debounce.v rtl/sensor_veiculo.v rtl/botao_pedestre.v rtl/prescaler.v
rtl/contador_tempo.v rtl/fsm_semaforo.v rtl/estado_basico_decoder.v rtl/bram_historico.v rtl/media_move1_dsp.v rtl/pro
tocolo_serial.v rtl/top_semaforo.v; synth_gowin -top top_semaforo -json build/top_semaforo.json"
nextpnr-himbaechel --device GW1NSR-LV4CQN48PC7/I6 --vopt family=GW1NS-4 \
Info:          IOB:      12/   124    9%
Info:          LUT4:    455/  4608    9%
Info:          DFF:     151/  3456    4%
Info:          BSRAM:     1/    10   10%
Info:          MULT18X18: 1/    16    6%
Info: Max frequency for clock 'u_botao.clk': 218.82 MHz (PASS at 27.00 MHz)
Info: Max frequency for clock 'u_botao.clk': 198.69 MHz (PASS at 27.00 MHz)
gowin_pack -d GW1NS-4 -o build/top_semaforo.fs build/top_semaforo_pnr.json
$ ls -l build/top_semaforo.fs
-rw-rw-r-- 1 lucas-pordeus lucas-pordeus 1771710 Oct  6 23:32 build/top_semaforo.fs
```

Fonte: elaborado pelo autor (2026).

Tabela 12 – Utilização da FPGA

Recurso	Usado	Disponível	Uso
IOB	12	124	9 %
LUT4	455	4 608	9 %
DFF	151	3 456	4 %
BSRAM	1	10	10 %
MULT18X18 (DSP)	1	16	6 %
PLL	0	2	0 %
Frequência máxima	198,69 MHz	exigido 27 MHz	folga de 7,4×

Fonte: elaborado pelo autor (2026).

5.15 Software ARM

O código ARM foi montado e ligado com `--fatal-warnings` (qualquer aviso interromperia o build). O binário é um ELF AArch64 estático. Fora da Raspberry Pi, sob `qemu-aarch64`, o programa não encontra `/dev/gpiomem` e termina com mensagem de erro e código 1 — o comportamento correto, já que não existe modo simulado.

Figura 47 – Montagem e execução do monitor fora da Raspberry Pi

```
terminal · montagem do monitor ARM64

$ rm -rf build/lib build/monitor_semaforo*
$ make asm AS="aarch64-linux-gnu-as --fatal-warnings" LD="aarch64-linux-gnu-ld --fatal-warnings" AR=aarch64-linux-gnu-ar
aarch64-linux-gnu-as --fatal-warnings -g -o build/monitor_semaforo.o asm/monitor_semaforo.s
aarch64-linux-gnu-as --fatal-warnings -g -o build/lib/gpio_lib.o asm/lib/gpio_lib.s
aarch64-linux-gnu-as --fatal-warnings -g -o build/lib/strings_lib.o asm/lib/strings_lib.s
aarch64-linux-gnu-as --fatal-warnings -g -o build/lib/protocolo_serial_gpio.o asm/lib/protocolo_serial_gpio.s
aarch64-linux-gnu-as --fatal-warnings -g -o build/lib/telemetria.o asm/lib/telemetria.s
aarch64-linux-gnu-ar rcs build/lib/libembarcado.a build/lib/gpio_lib.o build/lib/strings_lib.o build/lib/protocolo_serial_gpio.o build/lib/telemetria.o
aarch64-linux-gnu-ld --fatal-warnings -o build/monitor_semaforo build/monitor_semaforo.o build/lib/libembarcado.a
$ file build/monitor_semaforo
build/monitor_semaforo: ELF 64-bit LSB executable, ARM aarch64, version 1 (SYSV), statically linked, with debug_info, not stripped
$ qemu-aarch64 build/monitor_semaforo; echo "codigo de saida: $?"
Monitor do semaforo (telemetria da Tang Nano 4K, Ctrl+C encerra) ==
ERRO: nao foi possivel mapear /dev/gpiomem (rode na Raspberry Pi com sudo)
codigo de saida: 1
```

Fonte: elaborado pelo autor (2026).

A figura seguinte mostra o objdump da rotina de validação: a sequência ubfx/cmp/b.cond, tbz e cbnz exatamente como escrita no fonte.

Figura 48 – Desmontagem de telemetria_valida

```
terminal · objdump de telemetria_valida

$ aarch64-linux-gnu-objdump -d build/lib/telemetria.o | sed -n "/<telemetria_valida>:/,/^$/p"
0000000000000000 <telemetria_valida>:
0: 530c3c01 ubfx w1, w0, #12, #4
4: 7100283f cmp w1, #0xa
8: 54000161 b.ne 34 <telemetria_valida+0x34> // b.any
c: 530a2c01 ubfx w1, w0, #10, #2
10: 71000c3f cmp w1, #0x3
14: 54000100 b.eq 34 <telemetria_valida+0x34> // b.none
18: 53061c02 ubfx w2, w0, #6, #2
1c: 71000c5f cmp w2, #0x3
20: 540000a0 b.eq 34 <telemetria_valida+0x34> // b.none
24: 36480040 tbz w0, #9, 2c <telemetria_valida+0x2c>
28: 35000061 cbnz w1, 34 <telemetria_valida+0x34>
2c: 52800020 mov w0, #0x1 // #1
30: d65f03c0 ret
34: 52800000 mov w0, #0x0 // #0
38: d65f03c0 ret
```

Fonte: elaborado pelo autor (2026).

Para garantir que a Raspberry interpreta corretamente **cada** pacote, a rotina telemetria_formata foi executada para os 65 536 quadros possíveis de 16 bits e a saída foi comparada com uma referência independente em Python: 1 536 quadros válidos, todos os demais rejeitados, **0 divergências**. O driver protocolo_serial_le_quadro foi testado contra um modelo do registrador de deslocamento da FPGA, recebendo corretamente 6 de 6 quadros, cada um com exatamente 16 bordas de sclk.

Figura 49 – Teste exaustivo do decodificador de telemetria

```
terminal · teste exaustivo do decodificador

$ qemu-aarch64 ./teste_formata | python3 compara_decodificador.py
quadros testados: 65536
quadros validos: 1536
divergencias: 0
0xA944 → carros=VERDE pedestres=VERMELHO restante=4s fluxo=MEDIO pedido=SIM
0xA207 → carros=VERMELHO pedestres=VERDE restante=7s fluxo=BAIXO pedido=NAO
0xA600 → QUADRO INVALIDO (valor=42496): confira fiacao, GND comum e se a FPGA esta gravada
0x0000 → QUADRO INVALIDO (valor=0): confira fiacao, GND comum e se a FPGA esta gravada
0xFFFF → QUADRO INVALIDO (valor=65535): confira fiacao, GND comum e se a FPGA esta gravada
```

Fonte: elaborado pelo autor (2026).

Figura 50 – Teste do driver serial contra o modelo da FPGA

```
terminal · teste do driver serial

$ qemu-aarch64 ./teste_driver # 6 quadros: A9C5 A001 0000 FFFF 5A3C A2BF
OK
OK
OK
OK
OK
OK
```

Fonte: elaborado pelo autor (2026).

5.16 Validação no hardware físico

Os testes acima validam o comportamento lógico, a síntese e o software. A validação na placa segue o roteiro da seção 7 de GUIA_MONTAGEM_HARDWARE.md, resumido na Tabela 13. A coluna de resultado deve ser preenchida na bancada, acompanhada das fotos e do vídeo de demonstração.

Tabela 13 – Roteiro de validação na placa

#	Ação	Resultado esperado	Resultado observado
A	ligar a Tang Nano	carro verde + pedestre vermelho	
B	esperar 1 minuto sem apertar	continua verde (sem pedido não há troca)	
C	sem veículos por 50 s, apertar S1	~5 s → amarelo 3 s → pedestre 15 s → verde	
D	passar a mão no sensor ~6×/10 s por 1 min, apertar S1	~20 s até o amarelo	
E	apertar S2 em qualquer fase	volta a carro verde + pedestre vermelho	
F	rodar make run na Raspberry	1 linha/s com as mesmas cores dos LEDs	
G	repetir C e D com o monitor	pedido=SIM; fluxo=ALTO; contagem regressiva	
H	desconectar o fio do miso	QUADRO INVALIDO; semáforo inalterado	

Fonte: elaborado pelo autor (2026).

6. HISTÓRICO DE DEPURAÇÃO

A Tabela 14 reúne os defeitos reais encontrados ao longo do projeto, com causa e correção. Os quatro últimos foram encontrados na revisão final para o hardware físico.

Tabela 14 – Defeitos encontrados e corrigidos

Sintoma	Causa	Correção
FSM carregava a duração errada na troca de fase (TP3)	contador carregado com a fase atual	carga pela fase de destino (prox_estado)
média nunca saía de “baixo” (TP4/TP5)	amostra fixa em 1 por veículo	amostra = veículos por janela de tempo
média atrasada e mal arredondada (TP4)	usava a soma antiga e truncava	soma nova e + 0,5 antes do deslocamento
uint_to_dec corrompia o retorno	dígitos gravados sobre o x30 salvo	registradores de 64 bits e dígitos em [sp+16]
placa não avançava	clk mapeado no pino 27	oscilador real no pino 45
fases de ~370 ns, invisíveis	FSM contava ciclos de clock	prescaler de 1 tick por segundo
BRAM removida pela síntese	gravava sempre 1 e ninguém lia a saída	passou a guardar as amostras lidas pela média
fluxo quase nunca mudava a espera	tempo de verde aplicado só ao entrar na fase	também vale a partir do aperto
debounce inútil para o botão	8 ciclos = 300 ns	20 ms (botão) e 5 ms (sensor)
sensor acima da tensão do banco	pinos do HDMI, banco de 2,5 V	sensor e LEDs nos bancos de 3,3 V

Fonte: elaborado pelo autor (2026).

7. RUBRICAS CONSOLIDADAS

Tabela 15 – Critérios da entrega final e evidências

Critério	Evidência	Situação
Entradas, saídas e fluxos explicados	seções 2.1 e 2.2; Figura 1; Tabela 2	atendido
Divisão de responsabilidades justificada	seção 2.3; Tabela 3	atendido
Tempo real: BRAM, DSP, PLL, banda	seção 2.4; Tabelas 4 e 12	atendido
Diagramas integrais	Figuras 1, 11 e 15	atendido
Combinacionais e testbenches corretos	seções 3 e 5; 10 testbenches, 62 verificações	atendido
Waveforms comprovando temporização	seções 5.2 a 5.13 (GTKWave)	atendido
FSMs modulares validadas no hardware físico	seção 3.5 e 5.11; roteiro da seção 5.16	simulação ok; bancada a registrar
Integração aritmética BRAM/DSP igual à simulação	seções 3.3, 3.4, 5.8 e 5.14	atendido em simulação e síntese
LDR/STR, registradores, decisões e loops	seção 4 (GPIO, driver, decodificador)	atendido
Vetores, strings e parsers	tabelas indexadas, uint_to_dec, parser de bitfields	atendido
NEON SIMD e cálculos avançados	TP4/asm/tp4_neon_simd.s (fora do sistema final)	atendido no TP4
Modularidade: arquivos, macros, biblioteca estática	libembarcado.a, macros de telemetria.s, Makefile	atendido
Sinais via GPIO mapeados	seção 4.1	atendido
Protocolo com bitfields e handshaking	seção 3.6; cs_n delimita e congela o quadro	atendido
Sincronização elétrica com periféricos	seção 2.2; bancos de tensão; sincronizadores	atendido; conferir na bancada
Detecção de erros e confiabilidade	marcador 1010 e regras de coerência; teste de 65 536 quadros	atendido
Prints, logs e waveforms organizados	seção 5 e docs/evidencias/	atendido
Testes unitários por subsistema	Tabela 11	atendido
Dados contínuos de desempenho e latência	latências de debounce, FSM e protocolo; timing de 198 MHz	medição contínua na placa a registrar

Fonte: elaborado pelo autor (2026).

8. CONCLUSÃO

O sistema final concentra na FPGA tudo o que tem requisito de tempo real e de segurança, e deixa à Raspberry Pi apenas a observação do estado por um link serial somente de leitura. A revisão para o hardware físico corrigiu problemas que a simulação anterior escondia — pinos em banco de tensão errado, debounce de nanossegundos, BRAM removida pela síntese e um fluxo que quase nunca alterava a espera do pedestre — e removeu todo dado simulado do software.

A validação cobre os 11 módulos com 10 testbenches auto-verificáveis (62 verificações), a análise de cada forma de onda no GTKWave, a síntese e o place & route sem warnings com BRAM e DSP confirmados, e o software ARM montado sem warnings, com o decodificador conferido em todos os 65 536 quadros possíveis. O passo restante é registrar na bancada o roteiro da seção 5.16, com fotos e vídeo.

9. REPOSITÓRIO E VÍDEOS

Repositório Git:

https://github.com/LucasPordeus/lucas_gabriel_sistemas_digitais_embarcados

Vídeos do projeto:

https://drive.google.com/drive/folders/12veErgy4M6ZnE5FjXudPlkcf83e5Kqnh?usp=drive_link

Obs.: acesso liberado ao e-mail dacio.souza@prof.infnet.edu.br.